Proiect Baze de Date
Sistem de gestionare a unui Hackathon

Alexandra Pintea

Departamentul de Informatică,
Facultatea de Matematică și Informatică,
Universitatea de Vest, Timișoara, România
Email: alexandra.pintea06@e-uvt.ro

15 ianuarie 2026

Cuprins

1 Introducere **5**

2 Descrierea bazei de date **5**

3 Identificarea dependențelor funcționale și normalizarea **6**
3.1 Tabelul `Participants` 6
3.2 Tabelul `Themes` 6
3.3 Tabelul `Teams` 7
3.4 Tabelul `Ideas` 7
3.5 Tabelul `Judges` 7
3.6 Tabelul `Evaluations` 8

4 Diagrama Entitate-Relație (ERD) **8**
4.1 Relații 9
4.2 Rezumatul cheilor externe (FK) 9

5 Definirea tabelelor si exemplificarea comenzilor **10**
5.1 Ștergerea tabelelor existente 10
5.2 Crearea tabelului `Themes` 11
5.3 Crearea tabelului `Participants` 11
5.4 Crearea tabelului `Teams` 12
5.5 Crearea tabelului `Ideas` 13
5.6 Crearea tabelului `Judges` 14
5.7 Crearea tabelului `Evaluations` 15
5.8 Exemplificare de comenzi: diverse operații asupra tabelelor 16
5.8.1 Adăugarea unei noi coloane în tabel (ALTER) 16
5.8.2 Redenumirea unui tabel (RENAME) 17
5.8.3 Golirea unui tabel (TRUNCATE) 17
5.8.4 Ștergerea unui tabel (DROP TABLE) 18

6 Confirmarea existenței tabelelor și vizualizarea structurii și a constrângerilor **19**
6.1 Confirmarea existenței tabelelor în baza de date 19
6.2 Structura tabelului `Themes` 19
6.3 Structura tabelului `Participants` 20
6.4 Structura tabelului `Teams` 20
6.5 Structura tabelului `Ideas` 20
6.6 Structura tabelului `Judges` 21
6.7 Structura tabelului `Evaluations` 21
6.8 Constrângerile din tabelul `Themes` 21
6.9 Constrângerile din tabelul `Participants` 22
6.10 Constrângerile din tabelul `Teams` 22
6.11 Constrângerile din tabelul `Ideas` 23
6.12 Constrângerile din tabelul `Judges` 24
6.13 Constrângerile din tabelul `Evaluations` 24

7 Definirea de obiecte ale bazei de date, altele decât tabele **25**
7.1 Definirea vederilor . . . 25
7.1.1 Crearea vederii `v_teams_details` . . . 25
7.1.2 Crearea vederii `v_team_scores` . . . 26
7.1.3 Confirmarea existenței vederilor . . . 27
7.1.4 Ștergerea unei vederi . . . 27
7.1.5 Confirmarea inexistenței vederii . . . 27
7.2 Definirea secvențelor . . . 28
7.2.1 Crearea secvenței `seq_participant_id` . . . 28
7.2.2 Confirmarea existenței secvenței . . . 29
7.2.3 Modificarea secvenței . . . 29
7.2.4 Ștergerea secvenței . . . 29
7.2.5 Confirmarea inexistenței secvenței . . . 30
7.3 Definirea sinonimelor . . . 30
7.3.1 Crearea sinonimului `teams_syn` . . . 30
7.3.2 Confirmarea existenței sinonimului . . . 31
7.3.3 Ștergerea sinonimului . . . 31
7.3.4 Confirmarea inexistenței sinonimului . . . 32
7.4 Definirea indecșilor . . . 32
7.4.1 Crearea indexului `idx_part_nume` . . . 32
7.4.2 Confirmarea existenței indexului . . . 33
7.4.3 Ștergerea indexului . . . 33
7.4.4 Confirmarea inexistenței indexului . . . 33

8 Exemplificare de comenzi de prelucrare asupra datelor **34**
8.1 Adăugare de date (INSERT) . . . 34
8.2 Vizualizarea datelor din tabele după multiple inserări . . . 35
8.2.1 Datele din tabelul `Themes` . . . 36
8.2.2 Datele din tabelul `Participants` . . . 36
8.2.3 Datele din tabelul `Teams` . . . 36
8.2.4 Datele din tabelul `Ideas` . . . 37
8.2.5 Datele din tabelul `Judges` . . . 37
8.2.6 Datele din tabelul `Evaluations` . . . 37
8.3 Modificare de date (UPDATE) . . . 37
8.4 Ștergerea datelor (DELETE) . . . 38
8.5 Îmbinare (MERGE) . . . 39
8.6 Selecție (SELECT) – interogarea datelor . . . 40
8.6.1 SELECT simplu – afișarea temelor (Themes) . . . 41
8.6.2 SELECT simplu – afișarea participanților (Participants) . . . 41
8.6.3 SELECT cu filtrare (WHERE) – participanți cu studii Master . . . 41
8.6.4 SELECT cu JOIN – detalii echipe (Teams + Participants + Themes) 42
8.6.5 SELECT cu GROUP BY + funcții agregate – statistici evaluări pe echipe . . . 42

9 Selecția datelor – interogări complexe **43**
9.1 Interogarea 1: Echipe ordonate după media notelor (din Evaluations) . . . 43
9.2 Interogarea 2: Teme cu număr de echipe și scor mediu . . . 44
9.3 Interogarea 3: Liderii echipelor cu scor peste media generală . . . 44

9.4 Interogarea 4: Juraţi cu activitate ridicată şi note mari 45
9.5 Interogarea 5: Idei cu tehnologii specifice şi descriere suficient de lungă . 46

10 Interogări incluse în definiţia unei vederi (VIEW) 47
10.1 Crearea vederii `V_TOP_TEAMS_BY_AVG` 47
10.1.1 Confirmarea existenţei vederii în dicţionarul de date 47
10.2 Crearea vederii `V_THEME_STATS` 48
10.3 Crearea vederii `V_JUDGE_ACTIVITY` 49
10.4 Crearea vederii `V_TEAM_EVAL_RANK` 50

1 Introducere

Proiectul presupune realizarea unei **baze de date** pentru gestionarea unui eveniment de tip **Hackathon**, în care participanții pot propune idei, iar echipele își înscriu proiectele în cadrul unei teme. Baza de date urmărește organizarea informațiilor despre **participanți**, **echipe** și **idei/proiecte**, precum și organizarea echipelor participante, incluzând liderul echipei, tema aleasă și punctajul obținut. Scopul principal al bazei de date este:

- **înregistrarea participanților** (nume, prenume, email, etc.);
- **gestionarea echipelor** (nume echipă, lider, temă, punctaj final);
- **gestionarea temelor** disponibile în cadrul hackathonului;
- **gestionarea ideilor/proiectelor** (titlu, descriere, tehnologie/soft folosit, temă);
- **gestionarea evaluărilor** acordate de jurați echipelor participante.

Această structură relațională permite obținerea rapidă de rapoarte, de exemplu:

- **topul echipelor** cu cel mai mare punctaj;
- **ideile** dintr-o anumită temă (ex: "Green Energy");
- **jurații** care au oferit note peste un anumit prag (ex: ≥ 9.5).

Prin utilizarea bazei de date, organizatorii pot administra ușor evenimentul și pot genera rapoarte relevante pentru evaluare și analiză.

2 Descrierea bazei de date

Baza de date este concepută pentru a gestiona informațiile asociate unui eveniment de tip **Hackathon**, incluzând participanții, echipele, ideile propuse și procesul de evaluare. Baza de date include următoarele tabele principale, 6 la număr:

- **Participants**: stochează informații despre participanți, precum numele, prenumele, numărul de telefon, adresa de email și nivelul de studii.
- **Teams**: conține informații despre echipele participante, inclusiv numele echipei, liderul echipei, tema aleasă și punctajul final obținut.
- **Themes**: stochează temele disponibile în cadrul hackathonului (ex: Green Energy, Cultural, AI Innovations).
- **Ideas**: gestionează ideile propuse de echipe, incluzând titlul ideii, o scurtă descriere, software-ul utilizat și tema asociată.
- **Judges**: conține informații despre jurați, precum numele și adresa de email.
- **Evaluations**: stochează notele acordate de jurați echipelor participante.

Structura relațională a bazei de date permite obținerea unor rapoarte utile, precum:

- primele echipe cu cel mai mare punctaj;

- ideile aparținând unei anumite teme de hackathon;
- jurații care au acordat doar note peste un anumit prag.

Această bază de date oferă un suport eficient pentru organizarea și evaluarea unui eveniment de tip Hackathon, asigurând stocarea coerentă și accesul rapid la informațiile relevante.

3 Identificarea dependențelor funcționale și normalizarea

3.1 Tabelul `Participants`

- **Structură:**
  - `participant_id` (cheie primară)
  - `nume`, `prenume`, `nr_tel`, `email`, `studii` (coloane din tabel).
- **Dependențe funcționale:**

$$participant_id \rightarrow \{nume, prenume, nr_tel, email, studii\}$$

  Toate atributele depind complet și direct de cheia primară `participant_id`.
- **Normalizare:**
  - Nu există dependențe parțiale.
  - Nu există dependențe tranzitive.

  Concluzie: Tabelul `Participants` este deja în **BCNF**.

3.2 Tabelul `Themes`

- **Structură:**
  - `theme_id` (cheie primară)
  - `theme_name` (coloană din tabel).
- **Dependențe funcționale:**

$$theme_id \rightarrow \{theme_name\}$$

  Atributul depinde complet și direct de cheia primară `theme_id`.
- **Normalizare:**
  - Nu există dependențe parțiale.
  - Nu există dependențe tranzitive.

  Concluzie: Tabelul `Themes` este deja în **BCNF**.

3.3 Tabelul `Teams`

- **Structură:**
  - `team_id` (cheie primară)
  - `team_name`, `leader_id`, `theme_id`, `score` (coloane din tabel).
- **Dependențe funcționale:**

$$team_id \rightarrow \{team_name, leader_id, theme_id, score\}$$

  Toate atributele depind complet și direct de cheia primară `team_id`.
- **Normalizare:**
  - Nu există dependențe parțiale.
  - Nu există dependențe tranzitive.

  Concluzie: Tabelul `Teams` este deja în **BCNF**.

3.4 Tabelul `Ideas`

- **Structură:**
  - `idea_id` (cheie primară)
  - `team_id`, `theme_id`, `title`, `description`, `software` (coloane din tabel).
- **Dependențe funcționale:**

$$idea_id \rightarrow \{team_id, theme_id, title, description, software\}$$

  Toate atributele depind complet și direct de cheia primară `idea_id`.
- **Normalizare:**
  - Nu există dependențe parțiale.
  - Nu există dependențe tranzitive.

  Concluzie: Tabelul `Ideas` este deja în **BCNF**.

3.5 Tabelul `Judges`

- **Structură:**
  - `judge_id` (cheie primară)
  - `full_name`, `email` (coloane din tabel).
- **Dependențe funcționale:**

$$judge_id \rightarrow \{full_name, email\}$$

  Toate atributele depind complet și direct de cheia primară `judge_id`.

- **Normalizare:**
  - Nu există dependențe parțiale.
  - Nu există dependențe tranzitive.

  Concluzie: Tabelul `Judges` este deja în **BCNF**.

3.6 Tabelul `Evaluations`

- **Structură:**
  - `eval_id` (cheie primară)
  - `judge_id`, `team_id`, `score` (coloane din tabel).
- **Dependențe funcționale:**

$$eval_id \rightarrow \{judge_id, team_id, score\}$$

  Toate atributele depind complet și direct de cheia primară `eval_id`.

- **Normalizare:**
  - Nu există dependențe parțiale.
  - Nu există dependențe tranzitive.

  Concluzie: Tabelul `Evaluations` este deja în **BCNF**.

4 Diagrama Entitate-Relație (ERD)

Diagrama entitate-relație include următoarele componente:

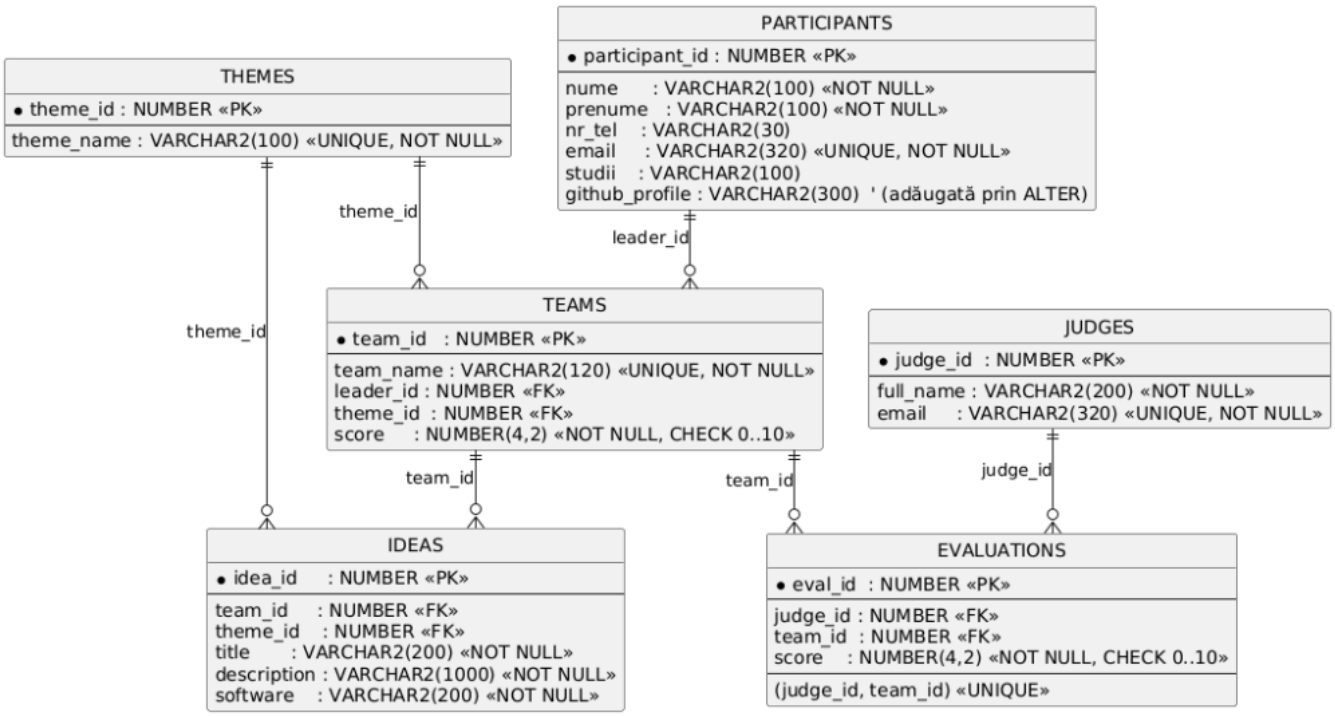


Diagrama ilustrează cele 6 tabele, atributele și tipurile de date asociate, precum și relațiile dintre ele.

4.1 Relații

Pe baza cheilor externe definite în tabele, rezultă următoarele relații între entități:

- **Themes și Teams (1:M)**: O temă (`Themes`) poate fi asociată cu mai multe echipe (`Teams`), dar fiecare echipă aparține unei singure teme. Legătura se face prin cheia externă `Teams.theme_id` care referă `Themes.theme_id`.
- **Participants și Teams (1:M) prin leader_id**: Un participant (`Participants`) poate fi lider pentru mai multe echipe, dar fiecare echipă are un singur lider. Legătura se face prin cheia externă `Teams.leader_id` care referă `Participants.participant_id`.
- **Teams și Ideas (1:M)**: O echipă (`Teams`) poate propune mai multe idei (`Ideas`), dar fiecare idee aparține unei singure echipe. Legătura se face prin cheia externă `Ideas.team_id` care referă `Teams.team_id`.
- **Themes și Ideas (1:M)**: O temă (`Themes`) poate avea mai multe idei (`Ideas`), dar fiecare idee este asociată unei singure teme. Legătura se face prin cheia externă `Ideas.theme_id` care referă `Themes.theme_id`.
- **Judges și Evaluations (1:M)**: Un jurat (`Judges`) poate oferi mai multe evaluări (`Evaluations`), dar fiecare evaluare este oferită de un singur jurat. Legătura se face prin cheia externă `Evaluations.judge_id` care referă `Judges.judge_id`.
- **Teams și Evaluations (1:M)**: O echipă (`Teams`) poate primi mai multe evaluări (`Evaluations`), dar fiecare evaluare aparține unei singure echipe. Legătura se face prin cheia externă `Evaluations.team_id` care referă `Teams.team_id`.

Observație importantă (relație M:N): Prin tabela `Evaluations`, între `Judges` și `Teams` există o relație de tip **M:N** (mulți-la-mulți), deoarece:

- un jurat poate evalua mai multe echipe;
- o echipă poate fi evaluată de mai mulți jurați.

Această relație M:N este implementată prin tabela `Evaluations`, care joacă rolul de **entitate asociativă**. În plus, constrângerea `UNIQUE(judge_id, team_id)` impune regula: un jurat poate evalua o echipă o singură dată.

4.2 Rezumatul cheilor externe (FK)

Pentru claritate, cheile externe utilizate în proiect sunt:

- `Teams.leader_id` → `Participants.participant_id`
- `Teams.theme_id` → `Themes.theme_id`
- `Ideas.team_id` → `Teams.team_id`

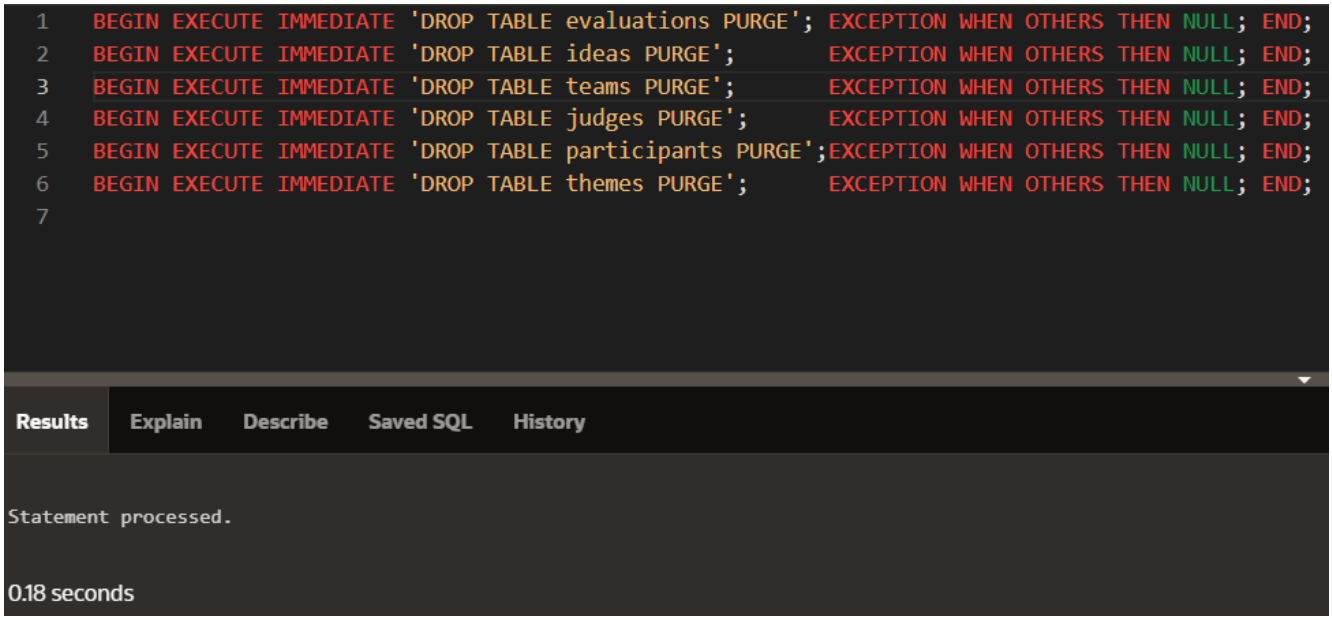
- `Ideas.theme_id` → `Themes.theme_id`
- `Evaluations.judge_id` → `Judges.judge_id`
- `Evaluations.team_id` → `Teams.team_id`

5 Definirea tabelelor si exemplificarea comenzilor

5.1 Ștergerea tabelelor existente

Înainte de definirea noii structuri a bazei de date, a fost necesară eliminarea tabelelor existente din schema curentă. Acest pas are rolul de a asigura faptul că operațiile de creare a tabelelor se realizează într-un mediu curat, fără influențe din definiții anterioare. Astfel, se evită apariția unor probleme sau erori.

```
BEGIN EXECUTE IMMEDIATE 'DROP TABLE evaluations PURGE'; EXCEPTION
    WHEN OTHERS THEN NULL; END;
BEGIN EXECUTE IMMEDIATE 'DROP TABLE ideas PURGE';       EXCEPTION
    WHEN OTHERS THEN NULL; END;
BEGIN EXECUTE IMMEDIATE 'DROP TABLE teams PURGE';       EXCEPTION
    WHEN OTHERS THEN NULL; END;
BEGIN EXECUTE IMMEDIATE 'DROP TABLE judges PURGE';      EXCEPTION
    WHEN OTHERS THEN NULL; END;
BEGIN EXECUTE IMMEDIATE 'DROP TABLE participants PURGE';EXCEPTION
    WHEN OTHERS THEN NULL; END;
BEGIN EXECUTE IMMEDIATE 'DROP TABLE themes PURGE';      EXCEPTION
    WHEN OTHERS THEN NULL; END;
```



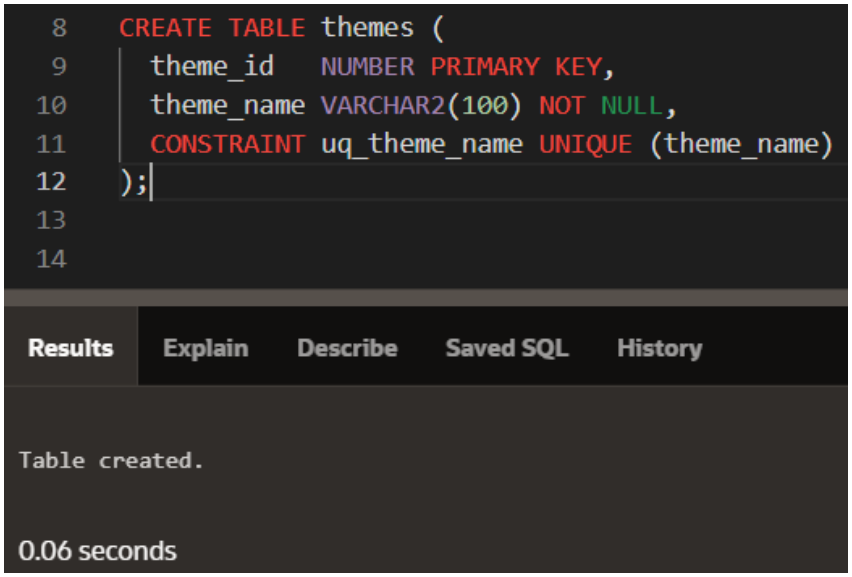
Imaginea 1: Ștergerea tabelelor existente (golire)

5.2 Crearea tabelului `Themes`

Tabelul `Themes` stochează lista de teme disponibile în cadrul hackathonului (de exemplu: Green Energy, Cultural, AI Innovations).

- **Cheia primară**: `theme_id`.
- **Coloane**: `theme_name` (numele temei).
- **Constrângeri**: `UNIQUE(theme_name)` – nu permite introducerea aceleiași teme de mai multe ori.

```sql
CREATE TABLE themes (
  theme_id   NUMBER PRIMARY KEY,
  theme_name VARCHAR2(100) NOT NULL,
  CONSTRAINT uq_theme_name UNIQUE (theme_name)
);
```



Imaginea 2: Crearea tabelului Themes

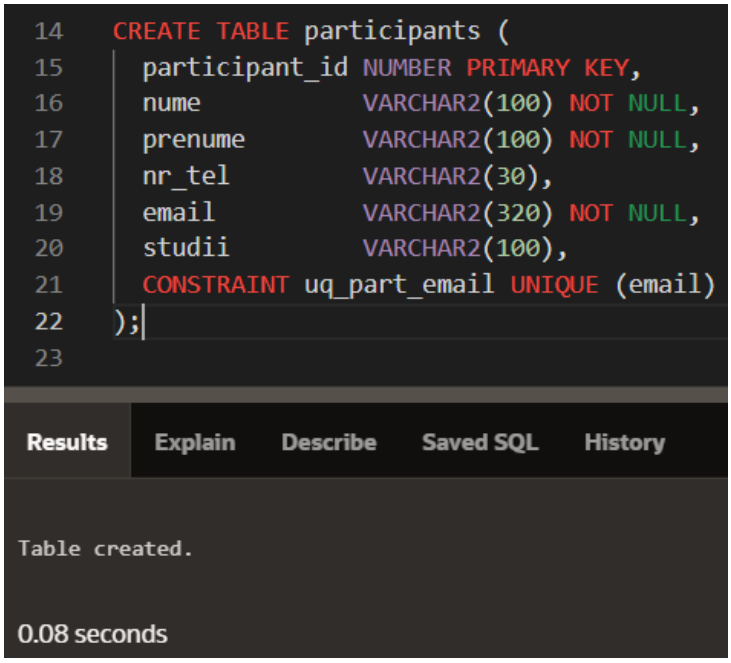
5.3 Crearea tabelului `Participants`

Tabelul `Participants` stochează informații despre participanții înscriși la hackathon.

- **Cheia primară**: `participant_id`.
- **Coloane**: `nume`, `prenume`, `nr_tel`, `email`, `studii`.
- **Constrângeri**: `UNIQUE(email)` – fiecare participant are o adresă de email unică.

```sql
CREATE TABLE participants (
  participant_id NUMBER PRIMARY KEY,
  nume           VARCHAR2(100) NOT NULL,
  prenume        VARCHAR2(100) NOT NULL,
  nr_tel         VARCHAR2(30),
```

```
  email           VARCHAR2(320) NOT NULL,
  studii          VARCHAR2(100),
  CONSTRAINT uq_part_email UNIQUE (email)
);
```



Imaginea 3: Crearea tabelului Participants

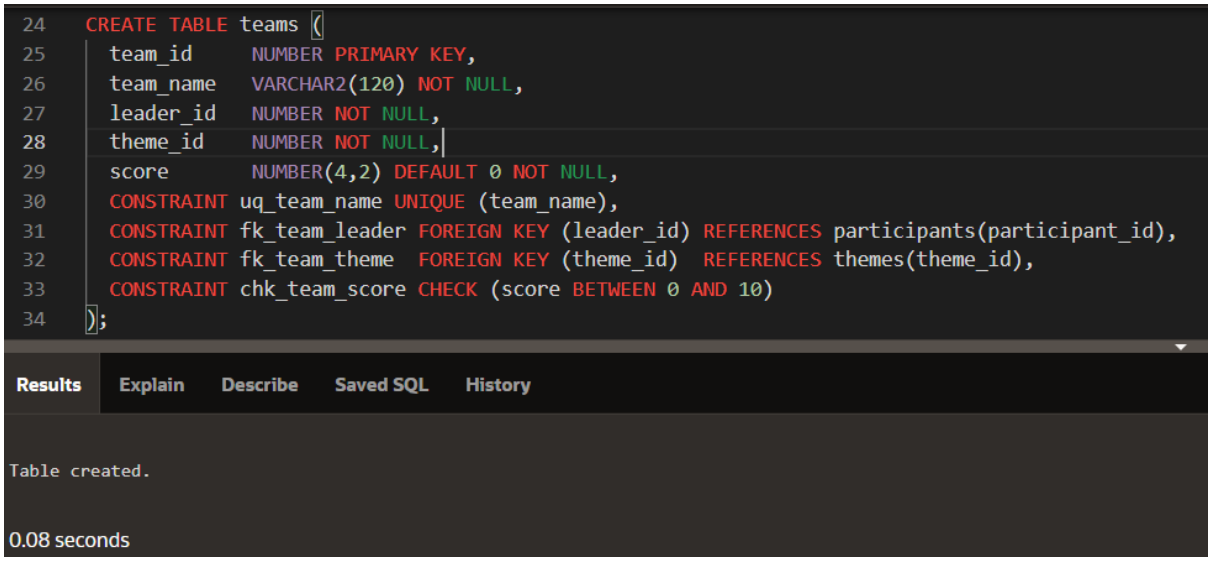
5.4 Crearea tabelului `Teams`

Tabelul `Teams` gestionează echipele participante în hackathon, inclusiv liderul echipei, tema aleasă și punctajul final.

- **Cheia primară**: `team_id`.
- **Chei externe**:
  - `leader_id` referă `participants(participant_id)` (liderul echipei).
  - `theme_id` referă `themes(theme_id)` (tema aleasă).
- **Coloane**: `team_name`, `score`.
- **Constrângeri**:
  - `UNIQUE(team_name)` – nu permite două echipe cu același nume.
  - `CHECK(score BETWEEN 0 AND 10)` – punctajul este limitat între 0 și 10.

```
CREATE TABLE teams (
  team_id      NUMBER PRIMARY KEY,
  team_name    VARCHAR2(120) NOT NULL,
  leader_id    NUMBER NOT NULL,
  theme_id     NUMBER NOT NULL,
```

```
  score        NUMBER(4,2) DEFAULT 0 NOT NULL,
  CONSTRAINT uq_team_name UNIQUE (team_name),
  CONSTRAINT fk_team_leader FOREIGN KEY (leader_id)
    REFERENCES participants(participant_id),
  CONSTRAINT fk_team_theme FOREIGN KEY (theme_id)
    REFERENCES themes(theme_id),
  CONSTRAINT chk_team_score CHECK (score BETWEEN 0 AND 10)
);
```



Imaginea 4: Crearea tabelului Teams

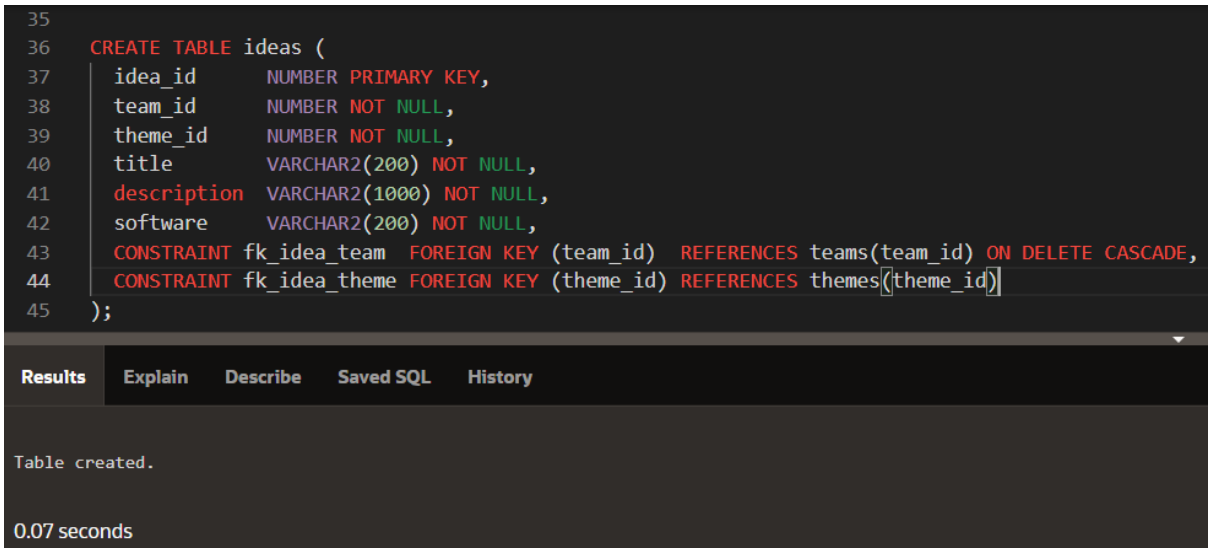
5.5 Crearea tabelului `Ideas`

Tabelul `Ideas` stochează ideile/proiectele propuse de echipe, împreună cu descrierea și tehnologia utilizată.

- **Cheia primară**: `idea_id`.
- **Chei externe**:
  - `team_id` referă `teams(team_id)`.
  - `theme_id` referă `themes(theme_id)`.
- **Coloane**: `title`, `description`, `software`.

```
CREATE TABLE ideas (
  idea_id       NUMBER PRIMARY KEY,
  team_id       NUMBER NOT NULL,
  theme_id      NUMBER NOT NULL,
  title         VARCHAR2(200) NOT NULL,
  description   VARCHAR2(1000) NOT NULL,
  software      VARCHAR2(200) NOT NULL,
  CONSTRAINT fk_idea_team  FOREIGN KEY (team_id)  REFERENCES
     teams(team_id),
```

```
  CONSTRAINT fk_idea_theme FOREIGN KEY (theme_id) REFERENCES
     themes(theme_id)
);
```



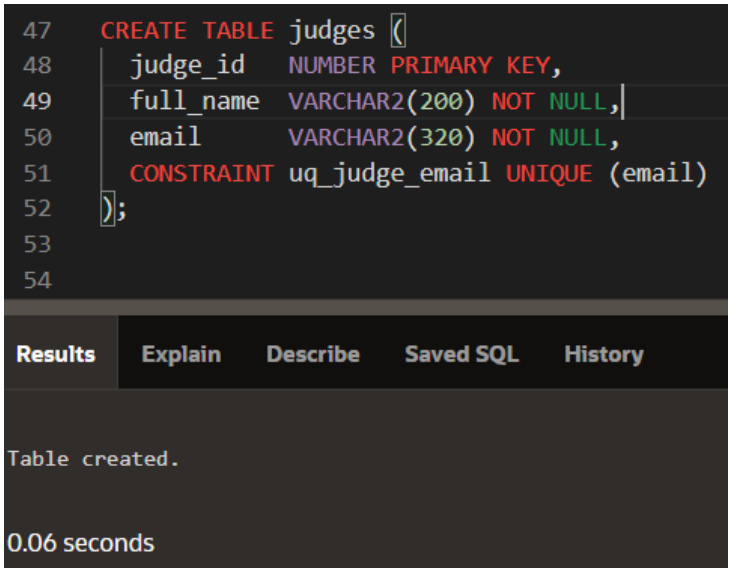
Imaginea 5: Crearea tabelului Ideas

5.6 Crearea tabelului `Judges`

Tabelul `Judges` stochează informații despre jurații hackathonului.

- **Cheia primară**: `judge_id`.
- **Coloane**: `full_name`, `email`.
- **Constrângeri**: `UNIQUE(email)` – fiecare jurat are un email unic.

```
CREATE TABLE judges (
  judge_id    NUMBER PRIMARY KEY,
  full_name   VARCHAR2(200) NOT NULL,
  email       VARCHAR2(320) NOT NULL,
  CONSTRAINT uq_judge_email UNIQUE (email)
);
```



Imaginea 6: Crearea tabelului Judges

5.7 Crearea tabelului `Evaluations`

Tabelul `Evaluations` stochează notele acordate de juraţi echipelor participante.

- **Cheia primară**: `eval_id`.
- **Chei externe**:
  - `judge_id` referă `judges(judge_id)`.
  - `team_id` referă `teams(team_id)`.
- **Coloane**: `score` (nota acordată).
- **Constrângeri**:
  - `CHECK(score BETWEEN 0 AND 10)` – nota este între 0 şi 10.
  - `UNIQUE(judge_id, team_id)` – un jurat poate nota o echipă o singură dată.

```sql
CREATE TABLE evaluations (
  eval_id    NUMBER PRIMARY KEY,
  judge_id   NUMBER NOT NULL,
  team_id    NUMBER NOT NULL,
  score      NUMBER(4,2) NOT NULL,
  CONSTRAINT fk_ev_judge FOREIGN KEY (judge_id) REFERENCES judges
     (judge_id),
  CONSTRAINT fk_ev_team  FOREIGN KEY (team_id)  REFERENCES teams(
     team_id),
  CONSTRAINT chk_ev_score CHECK (score BETWEEN 0 AND 10),
  CONSTRAINT uq_ev_judge_team UNIQUE (judge_id, team_id)
);
```

```
54  CREATE TABLE evaluations (
55    eval_id    NUMBER PRIMARY KEY,
56    judge_id   NUMBER NOT NULL,
57    team_id    NUMBER NOT NULL,
58    score      NUMBER(4,2) NOT NULL,
59    CONSTRAINT fk_ev_judge FOREIGN KEY (judge_id) REFERENCES judges(judge_id),
60    CONSTRAINT fk_ev_team  FOREIGN KEY (team_id)  REFERENCES teams(team_id),
61    CONSTRAINT chk_ev_score CHECK (score BETWEEN 0 AND 10),
62    CONSTRAINT uq_ev_judge_team UNIQUE (judge_id, team_id)
63  );
64
```

Results | Explain | Describe | Saved SQL | History

```
Table created.
```

0.07 seconds

Imaginea 7: Crearea tabelului Evaluations

5.8 Exemplificare de comenzi: diverse operații asupra tabelelor

5.8.1 Adăugarea unei noi coloane în tabel (ALTER)

Pentru a demonstra posibilitatea modificării structurii unui tabel existent, a fost utilizată comanda `ALTER TABLE`, prin care s-a adăugat o coloană nouă în tabela `Participants`. Această coloană este utilizată pentru stocarea link-ului către profilul GitHub al participantului.

```
ALTER TABLE participants
ADD (github_profile VARCHAR2(300));
```

```
65  ALTER TABLE participants ADD (github_profile VARCHAR2(300));
66
67
68
69
```

Results | Explain | Describe | Saved SQL | History

```
Table altered.
```

0.06 seconds

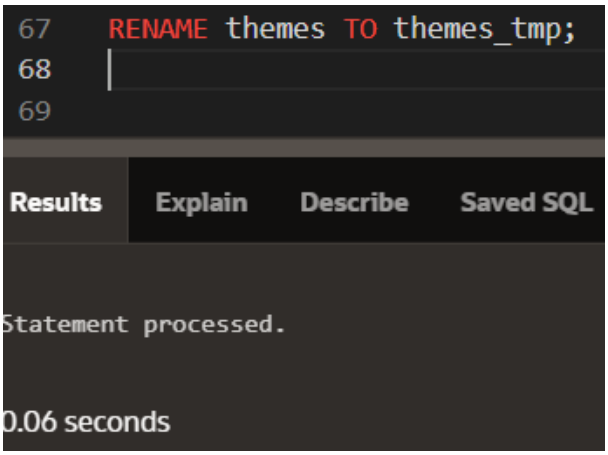
Imaginea 8: Modificarea structurii tabelului Participants

5.8.2 Redenumirea unui tabel (RENAME)

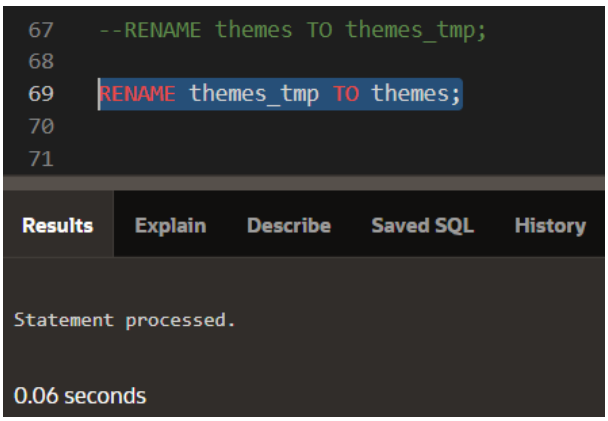
Pentru a exemplifica operația de redenumire a unui tabel, a fost utilizată comanda `RENAME`. Aceasta permite schimbarea numelui unui tabel existent, fără a afecta datele stocate în acesta.

```
RENAME themes TO themes_tmp;
```

```
RENAME themes_tmp TO themes;
```



Imaginea 9: Redenumirea tabelului Themes

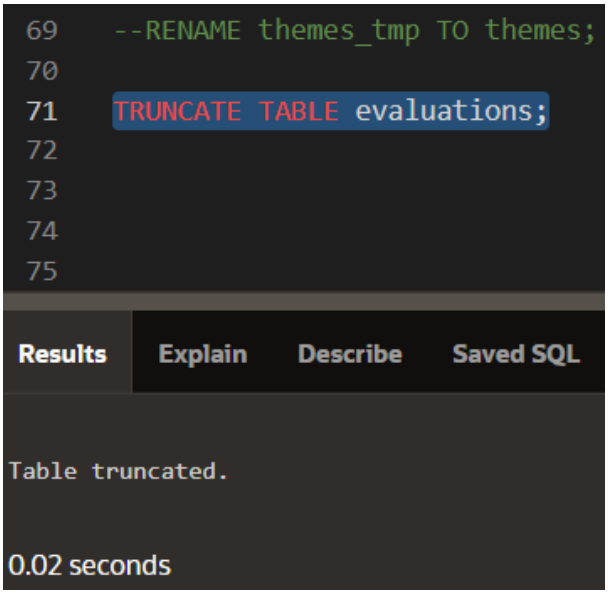


Imaginea 10: Aducerea tabelului Themes la numele inițial

5.8.3 Golirea unui tabel (TRUNCATE)

Comanda `TRUNCATE TABLE` este utilizată pentru a elimina toate înregistrările dintr-un tabel, fără a șterge structura acestuia. Această operație este mai rapidă decât `DELETE` și este utilizată atunci când se dorește resetarea conținutului tabelului.

```
TRUNCATE TABLE evaluations;
```

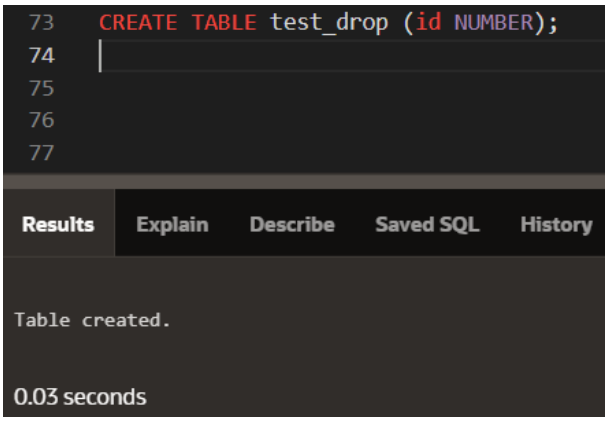


Imaginea 11: Golirea tabelului Evaluations

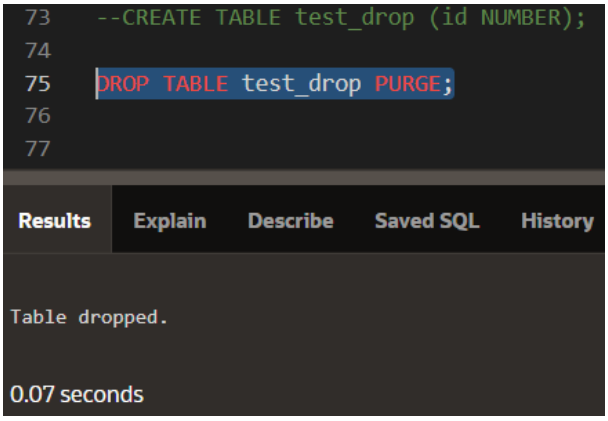
5.8.4 Ștergerea unui tabel (DROP TABLE)

Pentru a elimina complet un tabel din baza de date, inclusiv structura și datele sale, se utilizează comanda `DROP TABLE`. În acest proiect, această comandă a fost folosită pentru ștergerea tabelelor existente înainte de recrearea lor. Totodată, am exemplificat această comandă prin crearea unui nou tabel de test (pentru că nu vrem să le ștergem pe cele 6 cu care vom lucra), care a urmat a fi șters din baza de date:

```
CREATE TABLE test_drop (id NUMBER);
DROP TABLE test_drop PURGE;
```



Imaginea 12: Crearea unui nou tabel de test pentru ștergere



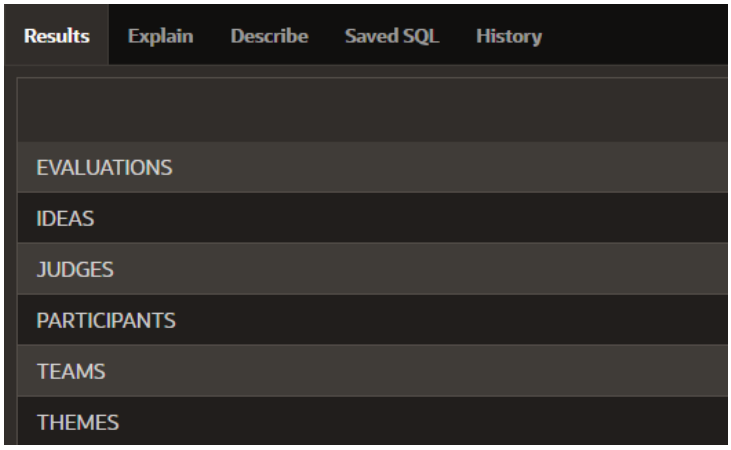
Imaginea 13: Ștergerea noului tabel de test creat

6 Confirmarea existenței tabelelor și vizualizarea structurii și a constrângerilor

6.1 Confirmarea existenței tabelelor în baza de date

Pentru a verifica tabelele existente în baza de date, am rulat următoarea comandă, care returnează o listă cu toate tabelele create în baza de date, ordonate lexicografic:

```sql
SELECT table_name
FROM user_tables
WHERE table_name IN ('THEMES','PARTICIPANTS','TEAMS','IDEAS','
    JUDGES','EVALUATIONS')
ORDER BY table_name;
```



Imaginea 14: Vizualizarea tabelelor existente în baza de date

6.2 Structura tabelului Themes

```sql
SELECT column_id, column_name, data_type, data_length, nullable
FROM user_tab_columns
WHERE table_name = 'THEMES'
ORDER BY column_id;
```

Această comandă afișează informațiile despre coloanele din tabelul `Themes`.

COLUMN_ID	COLUMN_NAME	DATA_TYPE	DATA_LENGTH	NULLABLE
1	THEME_ID	NUMBER	22	N
2	THEME_NAME	VARCHAR2	100	N

Imaginea 15: Detaliile din tabelul Themes

6.3 Structura tabelului `Participants`

```
SELECT column_id, column_name, data_type, data_length, nullable
FROM user_tab_columns
WHERE table_name = 'PARTICIPANTS'
ORDER BY column_id;
```

Această comandă afișează informațiile despre coloanele din tabelul `Participants`.

COLUMN_ID	COLUMN_NAME	DATA_TYPE	DATA_LENGTH	NULLABLE
1	PARTICIPANT_ID	NUMBER	22	N
2	NUME	VARCHAR2	100	N
3	PRENUME	VARCHAR2	100	N
4	NR_TEL	VARCHAR2	30	Y
5	EMAIL	VARCHAR2	320	N
6	STUDII	VARCHAR2	100	Y
7	GITHUB_PROFILE	VARCHAR2	300	Y

Imaginea 16: Detaliile din Tabelul Participants

6.4 Structura tabelului `Teams`

```
SELECT column_id, column_name, data_type, data_length, nullable
FROM user_tab_columns
WHERE table_name = 'TEAMS'
ORDER BY column_id;
```

Această comandă afișează informațiile despre coloanele din tabelul `Teams`.

COLUMN_ID	COLUMN_NAME	DATA_TYPE	DATA_LENGTH	NULLABLE
1	TEAM_ID	NUMBER	22	N
2	TEAM_NAME	VARCHAR2	120	N
3	LEADER_ID	NUMBER	22	N
4	THEME_ID	NUMBER	22	N
5	SCORE	NUMBER	22	N

Imaginea 17: Detaliile din Tabelul Teams

6.5 Structura tabelului `Ideas`

```
SELECT column_id, column_name, data_type, data_length, nullable
FROM user_tab_columns
WHERE table_name = 'IDEAS'
ORDER BY column_id;
```

Această comandă afișează informațiile despre coloanele din tabelul `Ideas`.

COLUMN_ID	COLUMN_NAME	DATA_TYPE	DATA_LENGTH	NULLABLE
1	IDEA_ID	NUMBER	22	N
2	TEAM_ID	NUMBER	22	N
3	THEME_ID	NUMBER	22	N
4	TITLE	VARCHAR2	200	N
5	DESCRIPTION	VARCHAR2	1000	N
6	SOFTWARE	VARCHAR2	200	N

Imaginea 18: Detaliile din Tabelul Ideas

6.6 Structura tabelului `Judges`

```sql
SELECT column_id, column_name, data_type, data_length, nullable
FROM user_tab_columns
WHERE table_name = 'JUDGES'
ORDER BY column_id;
```

Această comandă afișează informațiile despre coloanele din tabelul `Judges`.

COLUMN_ID	COLUMN_NAME	DATA_TYPE	DATA_LENGTH	NULLABLE
1	JUDGE_ID	NUMBER	22	N
2	FULL_NAME	VARCHAR2	200	N
3	EMAIL	VARCHAR2	320	N

Imaginea 19: Detaliile din Tabelul Judges

6.7 Structura tabelului `Evaluations`

```sql
SELECT column_id, column_name, data_type, data_length, nullable
FROM user_tab_columns
WHERE table_name = 'EVALUATIONS'
ORDER BY column_id;
```

Această comandă afișează informațiile despre coloanele din tabelul `Evaluations`.

COLUMN_ID	COLUMN_NAME	DATA_TYPE	DATA_LENGTH	NULLABLE
1	EVAL_ID	NUMBER	22	N
2	JUDGE_ID	NUMBER	22	N
3	TEAM_ID	NUMBER	22	N
4	SCORE	NUMBER	22	N

Imaginea 20: Detaliile din Tabelul Evaluations

6.8 Constrângerile din tabelul `Themes`

Pentru a afișa constrângerile asociate tabelului `Themes`, am rulat următoarea comandă:

```sql
SELECT constraint_name, constraint_type
FROM user_constraints
WHERE table_name = 'THEMES';
```

Această comandă afișează toate constrângerile asociate tabelului `Themes`.

- **PRIMARY KEY (P)** – identifică unic fiecare temă, pe baza coloanei `theme_id`.

- **UNIQUE (U)** – nu permite introducerea aceleiași teme de mai multe ori (pe coloana `theme_name`).
- **CHECK (C)** – constrângere generată automat de Oracle (de obicei pentru `NOT NULL`), care impune ca `theme_name` să nu fie vid.

CONSTRAINT_NAME	CONSTRAINT_TYPE
SYS_C00191498537	C
SYS_C00191498538	P
UQ_THEME_NAME	U

Imaginea 21: Contrângerile din Tabelul Themes

6.9 Constrângerile din tabelul `Participants`

Pentru a afișa constrângerile asociate tabelului `Participants`, am rulat următoarea comandă:

```
SELECT constraint_name, constraint_type
FROM user_constraints
WHERE table_name = 'PARTICIPANTS';
```

Această comandă afișează toate constrângerile asociate tabelului `Participants`.

- **PRIMARY KEY (P)** – identifică unic fiecare participant, pe baza coloanei `participant_id`.
- **UNIQUE (U)** – impune unicitatea emailului: nu pot exista doi participanți cu același `email`.
- **CHECK (C)** – constrângeri generate automat de Oracle (de obicei pentru `NOT NULL`), care impun completarea câmpurilor obligatorii (ex: `nume`, `prenume`, `email`).

CONSTRAINT_NAME	CONSTRAINT_TYPE
SYS_C00191498545	C
SYS_C00191498546	C
SYS_C00191498547	C
SYS_C00191498638	P
UQ_PART_EMAIL	U

Imaginea 22: Contrângerile din Tabelul Participants

6.10 Constrângerile din tabelul `Teams`

Pentru a afișa constrângerile asociate tabelului `Teams`, am rulat următoarea comandă:

```
SELECT constraint_name, constraint_type
FROM user_constraints
WHERE table_name = 'TEAMS';
```

Această comandă afișează toate constrângerile asociate tabelului `Teams`.

- **PRIMARY KEY (P)** – identifică unic fiecare echipă, pe baza coloanei `team_id`.
- **FOREIGN KEY (R)** – asigură legătura dintre tabele:
  - `leader_id` trebuie să existe în `participants(participant_id)` (liderul echipei este un participant valid).
  - `theme_id` trebuie să existe în `themes(theme_id)` (fiecare echipă are o temă validă).
- **UNIQUE (U)** – nu permite două echipe cu același nume (`team_name` este unic).
- **CHECK (C)** – verifică reguli asupra datelor:
  - `score` trebuie să fie între 0 și 10 (nu se acceptă valori în afara intervalului).
  - Oracle poate avea și alte constrângeri de tip `C` generate automat pentru `NOT NULL`.

CONSTRAINT_NAME	CONSTRAINT_TYPE
FK_TEAM_LEADER	R
FK_TEAM_THEME	R
SYS_C00191498939	C
SYS_C00191498940	C
SYS_C00191498941	C
SYS_C00191498942	C
CHK_TEAM_SCORE	C

Imaginea 23: Contrângerile din Tabelul Teams

6.11 Constrângerile din tabelul `Ideas`

Pentru a afișa constrângerile asociate tabelului `Ideas`, am rulat următoarea comandă:

```
SELECT constraint_name, constraint_type
FROM user_constraints
WHERE table_name = 'IDEAS';
```

Această comandă afișează toate constrângerile asociate tabelului `Ideas`.

- **PRIMARY KEY (P)** – identifică unic fiecare idee/proiect, pe baza coloanei `idea_id`.
- **FOREIGN KEY (R)** – asigură relațiile cu alte tabele:
  - `team_id` trebuie să existe în `teams(team_id)` (ideea aparține unei echipe valide).
  - `theme_id` trebuie să existe în `themes(theme_id)` (ideea este asociată unei teme valide).
- **CHECK (C)** – include constrângeri generate automat de Oracle pentru câmpuri obligatorii (ex: `title`, `description`, `software` nu pot fi `NULL`).

CONSTRAINT_NAME	CONSTRAINT_TYPE
SYS_C00191499029	C
SYS_C00191499030	C
SYS_C00191499031	C
SYS_C00191499032	C
SYS_C00191499033	P
FK_IDEA_TEAM	R
FK_IDEA_THEME	R

Imaginea 24: Contrângerile din Tabelul Ideas

6.12 Constrângerile din tabelul `Judges`

Pentru a afișa constrângerile asociate tabelului `Judges`, am rulat următoarea comandă:

```
SELECT constraint_name, constraint_type
FROM user_constraints
WHERE table_name = 'JUDGES';
```

Această comandă afișează toate constrângerile asociate tabelului `Judges`.

- **PRIMARY KEY (P)** – identifică unic fiecare jurat, pe baza coloanei `judge_id`.
- **UNIQUE (U)** – nu permite ca doi jurați să aibă același email (`email` este unic).
- **CHECK (C)** – constrângeri generate automat (ex: câmpuri `NOT NULL` precum `full_name` și `email`).

CONSTRAINT_NAME	CONSTRAINT_TYPE
SYS_C00191499194	C
SYS_C00191499195	C
SYS_C00191499196	P
UQ_JUDGE_EMAIL	U

Imaginea 25: Contrângerile din Tabelul Judges

6.13 Constrângerile din tabelul `Evaluations`

Pentru a afișa constrângerile asociate tabelului `Evaluations`, am rulat următoarea comandă:

```
SELECT constraint_name, constraint_type
FROM user_constraints
WHERE table_name = 'EVALUATIONS';
```

Această comandă afișează toate constrângerile asociate tabelului `Evaluations`.

- **PRIMARY KEY (P)** – identifică unic fiecare evaluare, pe baza coloanei `eval_id`.
- **FOREIGN KEY (R)** – menține integritatea referențială:
  - `judge_id` trebuie să existe în `judges(judge_id)` (evaluarea este dată de un jurat valid).
  - `team_id` trebuie să existe în `teams(team_id)` (evaluarea este pentru o echipă validă).

- **UNIQUE (U)** – nu permite ca același jurat să evalueze aceeași echipă de mai multe ori (unicitate pe perechea `(judge_id, team_id)`).
- **CHECK (C)** – verifică faptul că nota `score` este în intervalul 0–10.

CONSTRAINT_NAME	CONSTRAINT_TYPE
SYS_C00191499172	C
SYS_C00191499203	C
CHK_EV_SCORE	C
SYS_C00191499205	P
UQ_EV_JUDGE_TEAM	U
FK_EV_JUDGE	R
FK_EV_TEAM	R

Imaginea 26: Contrângerile din Tabelul Evaluations

7 Definirea de obiecte ale bazei de date, altele decât tabele

7.1 Definirea vederilor

Vederile (*views*) sunt obiecte ale bazei de date care permit salvarea unei interogări sub forma unei tabele virtuale. Acestea nu stochează date propriu-zise, ci afișează rezultate obținute din una sau mai multe tabele existente, facilitând accesul și analiza informațiilor.

7.1.1 Crearea vederii `v_teams_details`

Pentru a afișa informații detaliate despre echipele participante la hackathon, a fost creată vederea `v_teams_details`. Aceasta combină date din tabelele `Teams`, `Participants` și `Themes`.

```sql
CREATE OR REPLACE VIEW v_teams_details AS
SELECT
    t.team_id,
    t.team_name,
    p.nume || ' ' || p.prenume AS leader_name,
    th.theme_name,
    t.score
FROM teams t
JOIN participants p ON p.participant_id = t.leader_id
JOIN themes th ON th.theme_id = t.theme_id;
```

Această vedere afișează:

- identificatorul echipei;
- numele echipei;
- numele complet al liderului echipei;
- tema aleasă;
- punctajul obținut de echipă.

```
63 CREATE OR REPLACE VIEW v_teams_details AS
64 SELECT
65     t.team_id,
66     t.team_name,
67     p.nume || ' ' || p.prenume AS leader_name,
68     th.theme_name,
69     t.score
70 FROM teams t
71 JOIN participants p ON p.participant_id = t.leader_id
72 JOIN themes th ON th.theme_id = t.theme_id;
```

Results Explain Describe Saved SQL History

View created.

0.03 seconds

Imaginea 27: Creare vedere 1

7.1.2 Crearea vederii `v_team_scores`

Pentru a obține statistici legate de evaluarea echipelor, a fost creată vederea `v_team_scores`, care calculează numărul de evaluări și media notelor pentru fiecare echipă.

```
1 CREATE OR REPLACE VIEW v_team_scores AS
2 SELECT
3     t.team_id,
4     t.team_name,
5     COUNT(e.eval_id) AS nr_evaluari,
6     AVG(e.score)     AS medie_note
7 FROM teams t
8 LEFT JOIN evaluations e ON e.team_id = t.team_id
9 GROUP BY t.team_id, t.team_name;
```

Această vedere permite analiza performanței echipelor pe baza evaluărilor primite.

```
81 CREATE OR REPLACE VIEW v_team_scores AS
82 SELECT
83     t.team_id,
84     t.team_name,
85     COUNT(e.eval_id) AS nr_evaluari,
86     AVG(e.score)     AS medie_note
87 FROM teams t
88 LEFT JOIN evaluations e ON e.team_id = t.team_id
89 GROUP BY t.team_id, t.team_name;
```

Results Explain Describe Saved SQL History

View created.

0.03 seconds

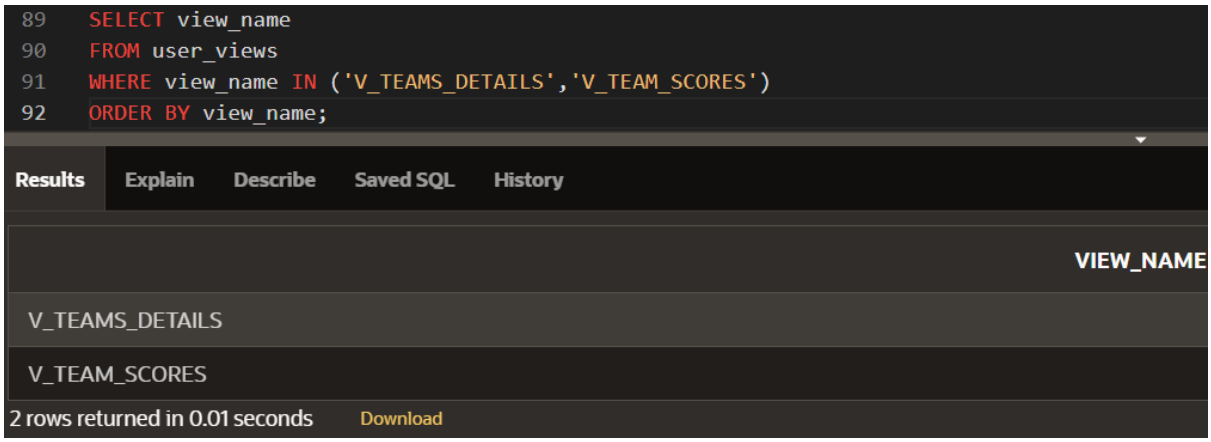
Imaginea 28: Creare vedere 2

7.1.3 Confirmarea existenței vederilor

Pentru a verifica existența vederilor create în baza de date, a fost utilizată următoarea interogare asupra dicționarului de date Oracle:

```
SELECT view_name
FROM user_views
WHERE view_name IN ('V_TEAMS_DETAILS','V_TEAM_SCORES')
ORDER BY view_name;
```

Această comandă returnează numele vederilor existente în schema curentă.



Imaginea 29: Vederile existente în schema curentă

7.1.4 Ștergerea unei vederi

Pentru a exemplifica eliminarea unei vederi, a fost utilizată comanda `DROP VIEW`. Pentru a evita apariția unei erori în cazul în care vederea nu există, am inclus comanda în blocul următor:

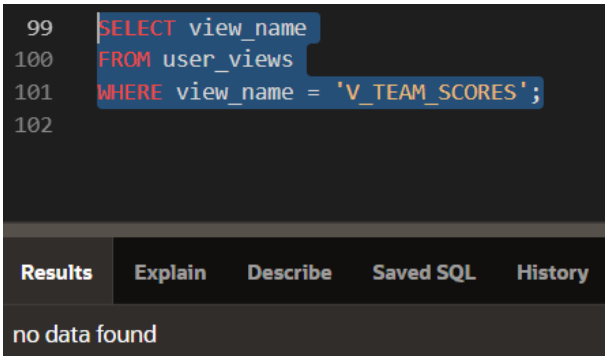
```
BEGIN
   EXECUTE IMMEDIATE 'DROP␣VIEW␣v_team_scores';
EXCEPTION
   WHEN OTHERS THEN NULL;
END;
```

7.1.5 Confirmarea inexistenței vederii

După ștergere, a fost verificată inexistența vederii, utilizând comanda:

```
SELECT view_name
FROM user_views
WHERE view_name = 'V_TEAM_SCORES';
```

Dacă interogarea nu returnează niciun rezultat, înseamnă că vederea a fost ștearsă cu succes, afișându-se mesajul "no data found".



Imaginea 30: Confirmare inexistență vedere

7.2 Definirea secvențelor

Secvențele (*sequences*) sunt obiecte ale bazei de date utilizate pentru generarea automată de valori numerice unice. Acestea sunt folosite, de regulă, pentru completarea cheilor primare, evitând introducerea manuală a identificatorilor.

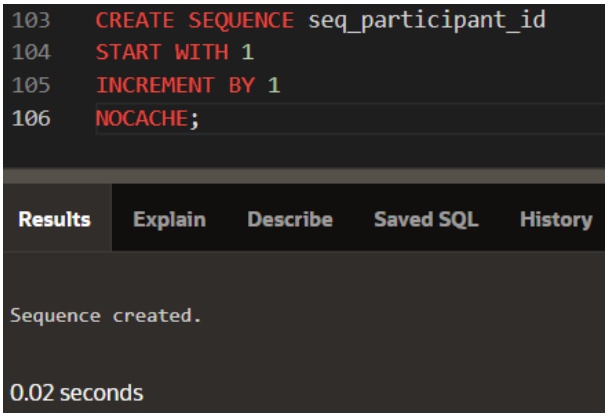
7.2.1 Crearea secvenței `seq_participant_id`

Pentru generarea automată a valorilor pentru identificatorul participanților, a fost creată secvența `seq_participant_id`.

```
CREATE SEQUENCE seq_participant_id
START WITH 1
INCREMENT BY 1
NOCACHE;
```

Această secvență:

- începe numerotarea de la valoarea 1;
- crește valoarea cu 1 la fiecare utilizare;
- nu utilizează mecanismul de cache.



Imaginea 31: Confirmare creare secvență

7.2.2 Confirmarea existenței secvenței

Pentru a verifica dacă secvența a fost creată cu succes, a fost interogat dicționarul de date Oracle utilizând comanda:

```
SELECT sequence_name
FROM user_sequences
WHERE sequence_name = 'SEQ_PARTICIPANT_ID';
```

Dacă secvența există, interogarea va returna numele acesteia.

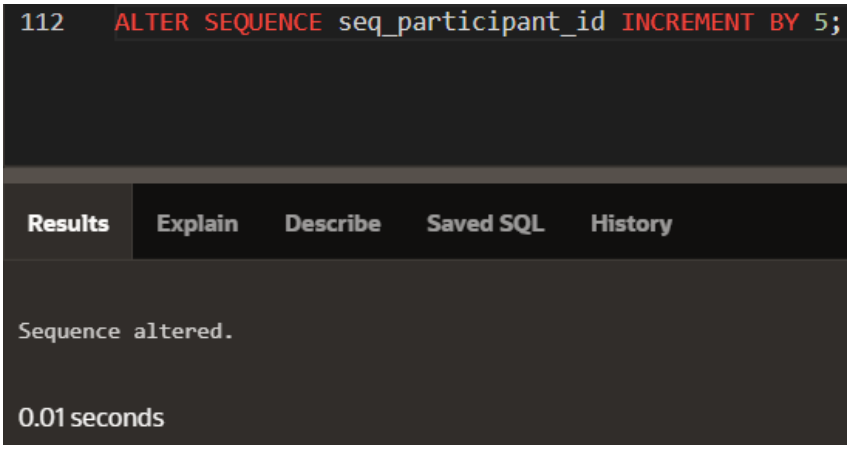
Imaginea 32: Confirmare existență secvență

7.2.3 Modificarea secvenței

Pentru a exemplifica modificarea unei secvențe, a fost utilizată comanda `ALTER SEQUENCE`, prin care a fost schimbată valoarea de increment.

```
ALTER SEQUENCE seq_participant_id
INCREMENT BY 5;
```

După această modificare, secvența va genera valori care cresc cu 5 la fiecare apelare.

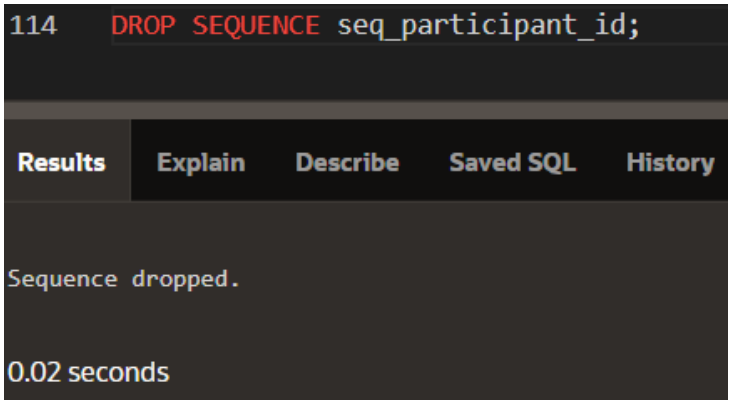


Imaginea 33: Confirmare alterare secvență

7.2.4 Ștergerea secvenței

Pentru a demonstra eliminarea unei secvențe din baza de date, a fost utilizată comanda `DROP SEQUENCE`.

```sql
DROP SEQUENCE seq_participant_id;
```

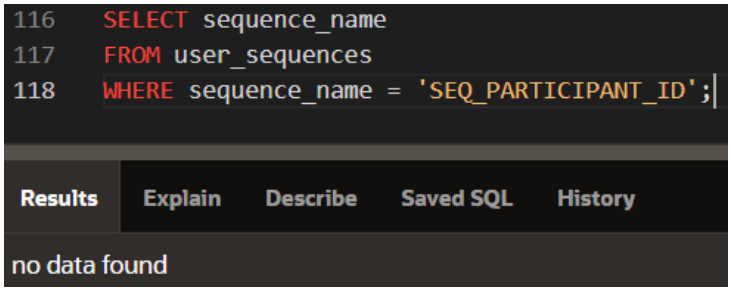


Imaginea 34: Ștergere secvență

7.2.5 Confirmarea inexistenței secvenței

După ștergere, a fost verificată inexistența secvenței utilizând interogarea:

```sql
SELECT sequence_name
FROM user_sequences
WHERE sequence_name = 'SEQ_PARTICIPANT_ID';
```



Imaginea 35: Confirmare inexistență secvență

Dacă această comandă nu returnează niciun rezultat (no data found), înseamnă că secvența a fost ștearsă cu succes.

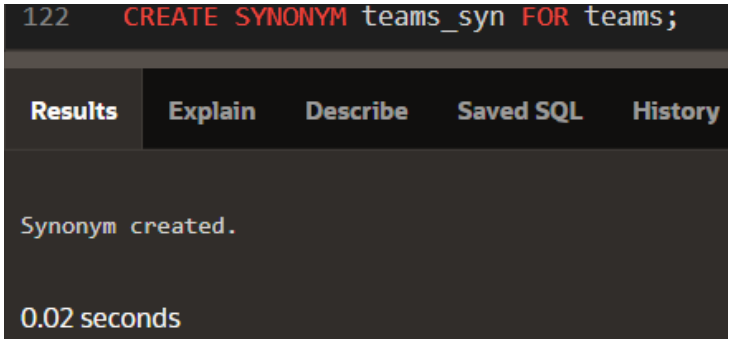
7.3 Definirea sinonimelor

Sinonimele (*synonyms*) sunt obiecte ale bazei de date care oferă un nume alternativ pentru un obiect existent (de exemplu, un tabel, o vedere, o secvență). Ele sunt utile pentru a scrie mai simplu numele unui tabel sau pentru a ascunde numele real al obiectului.

7.3.1 Crearea sinonimului `teams_syn`

În acest proiect, a fost creat un sinonim pentru tabelul `teams`, astfel încât acesta să poată fi referit și prin numele `teams_syn`. Am utilizat comanda:

```sql
CREATE SYNONYM teams_syn FOR teams;
```



Imaginea 36: Creare sinonim

După această comandă, tabelul `teams` poate fi accesat și astfel: `SELECT * FROM teams_syn;` (în loc de `SELECT * FROM teams;`).

7.3.2 Confirmarea existenței sinonimului

Pentru a confirma faptul că sinonimul a fost creat, a fost interogat dicționarul de date Oracle, folosind vederea `USER_SYNONYMS`:

```
SELECT synonym_name, table_name
FROM user_synonyms
WHERE synonym_name = 'TEAMS_SYN';
```

Dacă sinonimul există, interogarea va returna numele sinonimului și tabelul la care face referire.

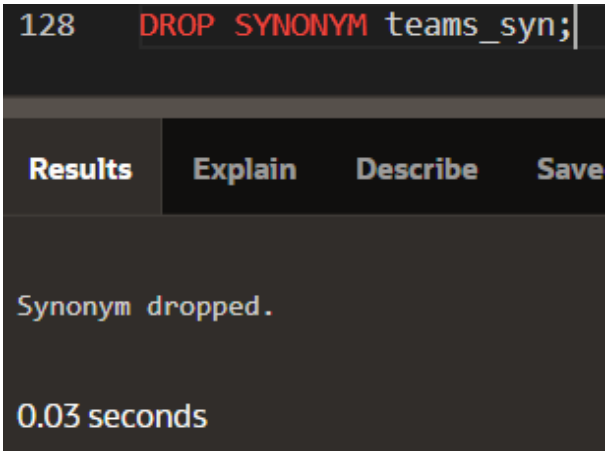
SYNONYM_NAME	TABLE_NAME
TEAMS_SYN	TEAMS

Imaginea 37: Confirmare existență sinonim

7.3.3 Ștergerea sinonimului

Pentru a demonstra eliminarea unui sinonim, a fost utilizată comanda:

```
DROP SYNONYM teams_syn;
```



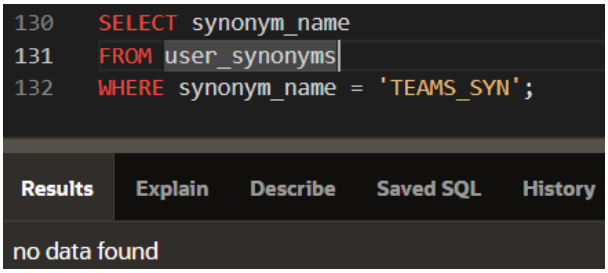
Imaginea 38: Ștergere sinonim

7.3.4 Confirmarea inexistenței sinonimului

După ștergere, a fost verificată inexistența sinonimului folosind interogarea:

```
SELECT synonym_name
FROM user_synonyms
WHERE synonym_name = 'TEAMS_SYN';
```

Dacă această comandă nu returnează niciun rezultat, înseamnă că sinonimul a fost șters cu succes (no data found).



Imaginea 39: Confirmare inexistență sinonim

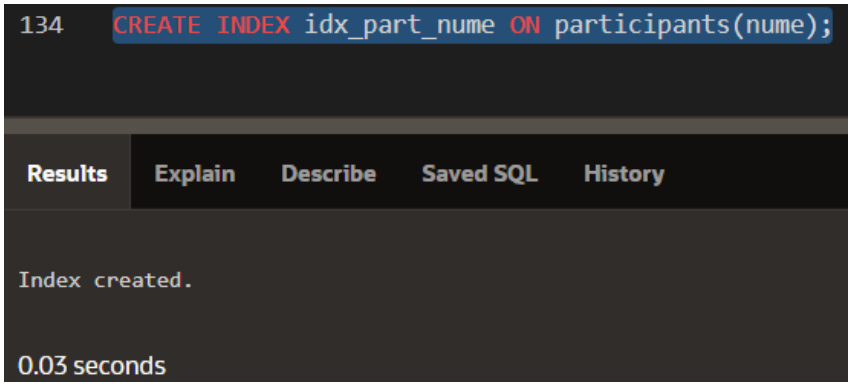
7.4 Definirea indecșilor

Indecșii (*indexes*) sunt obiecte ale bazei de date folosite pentru a îmbunătăți performanța interogărilor. Practic, indexul ajută baza de date să găsească mai rapid înregistrările, fără să parcurgă toate rândurile tabelului.

În acest proiect, a fost creat un index pe coloana `nume` din tabelul `participants`, pentru a accelera căutările de tipul: `WHERE nume = 'Popescu'`.

7.4.1 Crearea indexului `idx_part_nume`

```
CREATE INDEX idx_part_nume ON participants(nume);
```



Imaginea 40: Creare index

Acest index:

- este creat pe tabela `participants`;
- se bazează pe coloana `nume`;

- optimizează interogările care filtrează sau sortează după `nume`.

7.4.2 Confirmarea existenței indexului

Pentru a verifica dacă indexul a fost creat, a fost interogat dicționarul de date Oracle prin vederea `USER_INDEXES`:

```
SELECT index_name, table_name
FROM user_indexes
WHERE table_name = 'PARTICIPANTS';
```

INDEX_NAME	TABLE_NAME
IDX_PART_NUME	PARTICIPANTS
SYS_C00191498638	PARTICIPANTS
UQ_PART_EMAIL	PARTICIPANTS

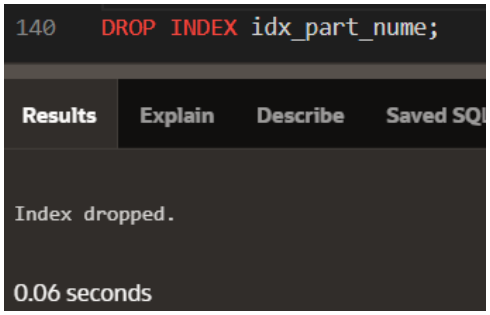
Imaginea 41: Confirmare existență index

Această comandă afișează toți indecșii existenți pentru tabelul `participants`.

7.4.3 Ștergerea indexului

Pentru a exemplifica eliminarea unui index, a fost utilizată comanda:

```
DROP INDEX idx_part_nume;
```



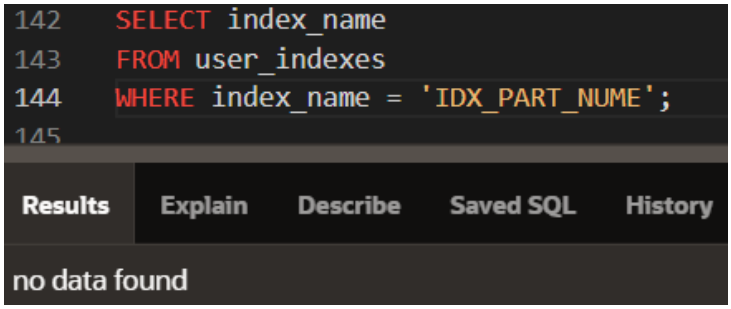
Imaginea 42: Ștergere index

7.4.4 Confirmarea inexistenței indexului

După ștergere, am verificat că indexul nu mai există, folosind:

```
SELECT index_name
FROM user_indexes
WHERE index_name = 'IDX_PART_NUME';
```

Dacă interogarea nu returnează rezultate, înseamnă că indexul a fost șters cu succes (no data found).



Imaginea 43: Confirmare inexistență index

8 Exemplificare de comenzi de prelucrare asupra datelor

Această secțiune prezintă comenzi de tip DML (Data Manipulation Language) utilizate pentru prelucrarea datelor din baza de date: adăugare (INSERT), modificare (UPDATE), ștergere (DELETE), îmbinare (MERGE) și selecție (SELECT). Toate exemplele au fost rulate în Oracle APEX, iar după fiecare operație s-a realizat confirmarea rezultatului prin afișarea datelor.

8.1 Adăugare de date (INSERT)

Operația de INSERT este utilizată pentru a adăuga date noi într-un tabel al bazei de date. Prin această comandă, se creează înregistrări noi care respectă structura tabelului și toate constrângerile definite (chei primare, chei externe, constrângeri de tip UNIQUE sau CHECK).

Inserarea datelor este un pas esențial, deoarece permite popularea bazei de date cu informații reale, care pot fi ulterior modificate, șterse sau analizate prin interogări SQL. Pentru a insera date corect, este importantă ordinea tabelelor, astfel încât să fie respectate cheile externe: mai întâi se inserează în tabelele „părinte” (Themes, Participants), apoi în tabelele care depind de acestea (Teams, Ideas, Judges, Evaluations).

```
-- 1) THEMES
INSERT INTO themes (theme_id, theme_name) VALUES (1, 'AI␣
    Innovations');
INSERT INTO themes (theme_id, theme_name) VALUES (2, 'Green␣
    Energy');

-- 2) PARTICIPANTS
INSERT INTO participants (participant_id, nume, prenume, nr_tel,
    email, studii)
VALUES (1, 'Popescu', 'Andrei', '0711111111', 'andrei.
    popescu@gmail.com', 'Licenta');

INSERT INTO participants (participant_id, nume, prenume, nr_tel,
    email, studii)
VALUES (2, 'Ionescu', 'Maria', '0722222222', 'maria.ionescu@gmail
    .com', 'Master');

```

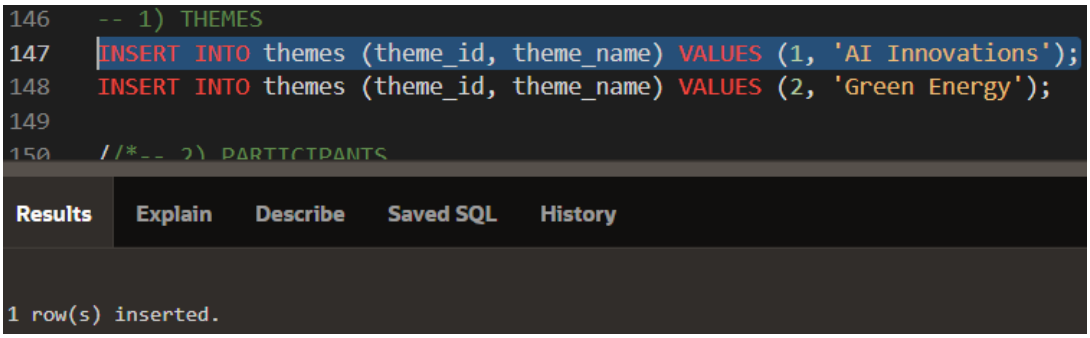
```sql
-- 3) TEAMS (are FK spre participants + themes)
INSERT INTO teams (team_id, team_name, leader_id, theme_id, score
    )
VALUES (1, 'HackMasters', 1, 1, 9.20);

-- 4) IDEAS (are FK spre teams + themes)
INSERT INTO ideas (idea_id, team_id, theme_id, title, description
    , software)
VALUES (1, 1, 1, 'Smart␣Mentor', 'Aplicatie␣AI␣pentru␣mentorat␣la
    ␣hackathon.', 'Python,␣FastAPI');

-- 5) JUDGES
INSERT INTO judges (judge_id, full_name, email)
VALUES (1, 'Dr.␣Mihai␣Stan', 'mihai.stan@uvt.ro');

-- 6) EVALUATIONS (are FK spre judges + teams)
INSERT INTO evaluations (eval_id, judge_id, team_id, score)
VALUES (1, 1, 1, 9.50);

COMMIT;
```



Imaginea 44: Confirmarea inserării datelor în tabelul Themes

8.2 Vizualizarea datelor din tabele după multiple inserări

În această subsecțiune sunt prezentate datele existente în fiecare tabel din proiect, pentru a demonstra că operațiile de inserare au fost realizate corect și că tabelele conțin înregistrări.

8.2.1 Datele din tabelul `Themes`

THEMES

Columns | Data | Indexes | Constraints | Grants | Statistics | Triggers | Dependencies | DDL | Sample Queries

+ Insert Row | Columns... | Filter... | Count Rows | Load Data | Download | Refresh

THEME_ID	THEME_NAME
1	AI Innovations
4	Healthcare Tech
3	Smart Cities
5	Education & Learning
2	Green Energy
10	Health Tech

Imaginea 45: Datele din tabelul `Themes`

8.2.2 Datele din tabelul `Participants`

PARTICIPANTS

Columns | Data | Indexes | Constraints | Grants | Statistics | Triggers | Dependencies | DDL | Sample Queries

+ Insert Row | Columns... | Filter... | Count Rows | Load Data | Download | Refresh

PARTICIPANT_ID	NUME	PRENUME	NR_TEL	EMAIL	STUDII	GITHUB_PROFILE
2	Ionescu	Maria	0799999999	maria.ionescu@gma...	Master	
3	Georgescu	Alex	0733333333	alex.georgescu@gm...	Licenta	
5	Marinescu	Radu	0755555555	radu.marinescu@g...	Doctorat	
4	Dumitrescu	Ioana	0744444444	ioana.dumitrescu@g...	Master	
1	Popescu	Andrei	0711111111	andrei.popescu@gm...	Licenta	

Imaginea 46: Datele din tabelul `Participants`

8.2.3 Datele din tabelul `Teams`

TEAMS

Columns | Data | Indexes | Constraints | Grants | Statistics | Triggers | Dependencies | DDL | Sample Queries

+ Insert Row | Columns... | Filter... | Count Rows | Load Data | Download | Refresh

TEAM_ID	TEAM_NAME	LEADER_ID	THEME_ID	SCORE
2	GreenFuture	2	2	8.75
3	CityCoders	3	3	9
4	HealthTechX	4	4	8.9
5	EduSmart	5	5	9.4

Imaginea 47: Datele din tabelul `Teams`

8.2.4 Datele din tabelul `Ideas`

IDEAS

Columns | Data | Indexes | Constraints | Grants | Statistics | Triggers | Dependencies | DDL | Sample Queries

+ Insert Row | Columns... | Filter... | Count Rows | Load Data | Download | Refresh

IDEA_ID	TEAM_ID	THEME_ID	TITLE	DESCRIPTION	SOFTWARE
2	2	2	Green Tracker	Monitorizarea consumul...	Java, Spring
4	4	4	HealthAI	Asistent medical bazat p...	Python, TensorFlow
5	5	5	EduPlatform	Platforma educationala i...	Angular, Firebase
3	3	3	CityFlow	Optimizarea traficului ur...	Node.js, React

Imaginea 48: Datele din tabelul `Ideas`

8.2.5 Datele din tabelul `Judges`

JUDGES

Columns | Data | Indexes | Constraints | Grants | Statistics | Triggers | Dependencies | DDL | Sample Queries

+ Insert Row | Columns... | Filter... | Count Rows | Load Data | Download | Refresh

JUDGE_ID	FULL_NAME	EMAIL
1	Dr. Mihai Stan	mihai.stan@uvt.ro
2	Prof. Ana Pop	ana.pop@uvt.ro
5	Dr. Elena Varga	elena.varga@uvt.ro
3	Dr. Radu Iliescu	radu.iliescu@uvt.ro
4	Conf. Ioan Matei	ioan.matei@uvt.ro

Imaginea 49: Datele din tabelul `Judges`

8.2.6 Datele din tabelul `Evaluations`

EVALUATIONS

Columns | Data | Indexes | Constraints | Grants | Statistics | Triggers | Dependencies | DDL | Sample Queries

+ Insert Row | Columns... | Filter... | Count Rows | Load Data | Download | Refresh

EVAL_ID	JUDGE_ID	TEAM_ID	SCORE
3	3	3	9.1
4	4	4	8.95
2	2	2	8.8
5	5	5	9.6

Imaginea 50: Datele din tabelul `Evaluations`

8.3 Modificare de date (UPDATE)

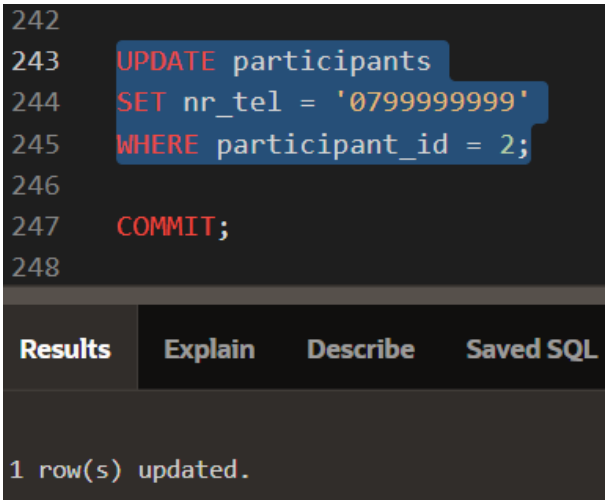
Comanda `UPDATE` este utilizată pentru modificarea valorilor existente într-un tabel. În exemplul de mai jos, se actualizează scorul unei echipe și numărul de telefon al unui

participant.

```
UPDATE teams
SET score = 9.60
WHERE team_id = 1;

UPDATE participants
SET nr_tel = '0799999999'
WHERE participant_id = 2;

COMMIT;
```



Imaginea 51: Update număr de telefon al unui participant

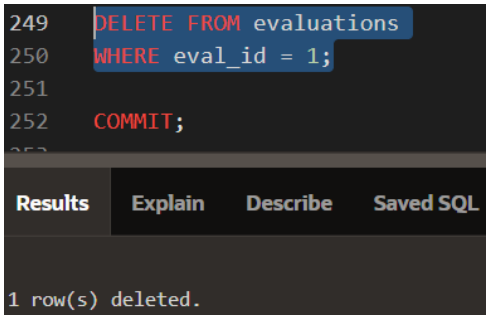
8.4 Ștergerea datelor (DELETE)

Operația `DELETE` este utilizată pentru eliminarea unor înregistrări din tabelele bazei de date, fără a afecta structura acestora. În cadrul proiectului, ștergerea a fost realizată respectând constrângerile de integritate referențială.

```
DELETE FROM evaluations
WHERE eval_id = 1;

COMMIT;
```

Comanda de mai sus elimină o evaluare din tabelul `evaluations`. După executarea comenzii, baza de date confirmă ștergerea prin mesajul *1 row(s) deleted*.



Imaginea 52: Eliminare evaluare

```
DELETE FROM ideas
WHERE idea_id = 1;

COMMIT;
```

Această comandă șterge o idee asociată unei echipe, demonstrând ștergerea controlată a datelor.

```
DELETE FROM teams
WHERE team_id = 1;

COMMIT;
```

În final, este eliminată o echipă din tabelul `teams`, după ce toate înregistrările dependente au fost șterse.

8.5 Îmbinare (MERGE)

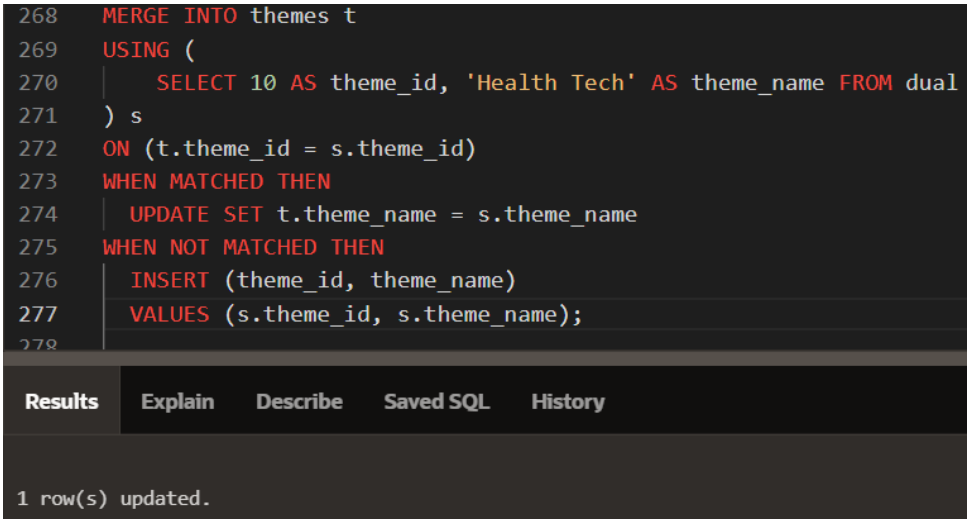
Instrucțiunea `MERGE` este utilizată pentru a combina într-o singură comandă operațiile de **inserare (INSERT)** și **actualizare (UPDATE)**. Aceasta verifică existența unui rând pe baza unei condiții și aplică acțiunea corespunzătoare în funcție de rezultat.

În exemplul următor, se verifică dacă există deja o temă cu `theme_id = 10` în tabelul `Themes`:

- dacă tema există, se actualizează numele acesteia;
- dacă tema nu există, aceasta este inserată în tabel.

```
MERGE INTO themes t
USING (
    SELECT 10 AS theme_id, 'Health Tech' AS theme_name FROM dual
) s
ON (t.theme_id = s.theme_id)
WHEN MATCHED THEN
    UPDATE SET t.theme_name = s.theme_name
WHEN NOT MATCHED THEN
    INSERT (theme_id, theme_name)
    VALUES (s.theme_id, s.theme_name);
```

În urma executării comenzii, Oracle afișează mesajul `1 row(s) updated`, ceea ce indică faptul că tema cu `theme_id = 10` exista deja în tabel și a fost actualizată.



Imaginea 53: Executare MERGE

Pentru confirmarea rezultatului, a fost utilizată următoarea interogare:

```
SELECT *
FROM themes
WHERE theme_id = 10;
```

Această interogare confirmă faptul că operația `MERGE` a fost executată cu succes, iar datele din tabelul `Themes` au fost actualizate corespunzător.

THEME_ID	THEME_NAME
10	Health Tech

rows returned in 0.01 seconds Download

Imaginea 54: Confirmare MERGE

8.6 Selecție (SELECT) – interogarea datelor

Operația `SELECT` este utilizată pentru **citirea / extragerea datelor** din tabelele bazei de date. Prin `SELECT` putem:

- să afișăm toate înregistrările dintr-un tabel;
- să filtrăm rezultatele folosind `WHERE`;
- să combinăm tabele folosind `JOIN`;
- să obținem statistici folosind funcții agregate (`COUNT`, `AVG`, etc.) împreună cu `GROUP BY`.

8.6.1 SELECT simplu – afișarea temelor (Themes)

Pentru a verifica rapid ce teme există în tabelul `Themes`, am rulat următoarea interogare:

```
SELECT theme_id, theme_name
FROM themes
ORDER BY theme_id;
```

Această comandă:

- selectează coloanele `theme_id` și `theme_name`;
- ordonează rezultatele crescător după `theme_id`.

THEME_ID	THEME_NAME
1	AI Innovations
2	Green Energy
3	Smart Cities
4	Healthcare Tech
5	Education & Learning
10	Health Tech

Imaginea 55: SELECT simplu din tabelul Themes

8.6.2 SELECT simplu – afișarea participanților (Participants)

Pentru a afișa lista participanților, am folosit:

```
SELECT participant_id, nume, prenume, email, studii
FROM participants
ORDER BY participant_id;
```

Această interogare:

- afișează participanții împreună cu datele lor principale;
- ajută la verificarea inserărilor realizate anterior.

PARTICIPANT_ID	NUME	PRENUME	EMAIL	STUDII
1	Popescu	Andrei	andrei.popescu@gmail.com	Licenta
2	Ionescu	Maria	maria.ionescu@gmail.com	Master
3	Georgescu	Alex	alex.georgescu@gmail.com	Licenta
4	Dumitrescu	Ioana	ioana.dumitrescu@gmail.com	Master
5	Marinescu	Radu	radu.marinescu@gmail.com	Doctorat

Imaginea 56: SELECT simplu din tabelul Participants

8.6.3 SELECT cu filtrare (WHERE) – participanți cu studii Master

Pentru a exemplifica filtrarea rezultatelor, am selectat doar participanții care au `studii = 'Master'`:

```
SELECT participant_id, nume, prenume, email
FROM participants
WHERE studii = 'Master';
```

PARTICIPANT_ID	NUME	PRENUME	EMAIL
2	Ionescu	Maria	maria.ionescu@gmail.com
4	Dumitrescu	Ioana	ioana.dumitrescu@gmail.com

Imaginea 57: SELECT simplu din tabelul Themes

Această comandă:

- folosește `WHERE` pentru a **filtra** datele;
- returnează doar participanții care îndeplinesc condiția impusă.

8.6.4 SELECT cu JOIN – detalii echipe (Teams + Participants + Themes)

Pentru a afișa detalii complete despre echipe (nume echipă, lider, tema și scor), am folosit o interogare cu `JOIN` între tabelele `Teams`, `Participants` și `Themes`:

```sql
SELECT
    t.team_id,
    t.team_name,
    p.nume || ' ' || p.prenume AS leader_name,
    th.theme_name,
    t.score
FROM teams t
JOIN participants p ON p.participant_id = t.leader_id
JOIN themes th ON th.theme_id = t.theme_id
ORDER BY t.score DESC;
```

Explicație:

- `JOIN participants` leagă echipa de liderul ei prin `leader_id`;
- `JOIN themes` leagă echipa de tema aleasă prin `theme_id`;
- expresia `p.nume || ' ' || p.prenume` construiește numele complet al liderului;
- `ORDER BY t.score DESC` sortează echipele descrescător după scor.

TEAM_ID	TEAM_NAME	LEADER_NAME	THEME_NAME	SCORE
5	EduSmart	Marinescu Radu	Education & Learning	9.4
3	CityCoders	Georgescu Alex	Smart Cities	9
4	HealthTechX	Dumitrescu Ioana	Healthcare Tech	8.9
2	GreenFuture	Ionescu Maria	Green Energy	8.75

Imaginea 58: SELECT cu JOIN: detalii despre echipe

8.6.5 SELECT cu GROUP BY + funcții agregate – statistici evaluări pe echipe

Pentru a exemplifica o selecție complexă (agregare), am calculat:

- numărul de evaluări pentru fiecare echipă (`COUNT`);
- media notelor (`AVG`).

```sql
SELECT
    t.team_id,
    t.team_name,
    COUNT(e.eval_id) AS nr_evaluari,
    ROUND(AVG(e.score), 2) AS medie_note
FROM teams t
LEFT JOIN evaluations e ON e.team_id = t.team_id
GROUP BY t.team_id, t.team_name
ORDER BY medie_note DESC;
```

Explicație:

- `LEFT JOIN` păstrează toate echipele, chiar dacă unele nu au evaluări;
- `COUNT(e.eval_id)` numără evaluările;
- `AVG(e.score)` calculează media notelor;
- `GROUP BY` este necesar pentru a agrega rezultatele pe fiecare echipă;
- `ROUND(..., 2)` rotunjește media la 2 zecimale.

TEAM_ID	TEAM_NAME	NR_EVALUARI	MEDIE_NOTE
5	EduSmart	1	9.6
3	CityCoders	1	9.1
4	HealthTechX	1	8.95
2	GreenFuture	1	8.8

Imaginea 59: SELECT cu GROUP BY: număr evaluări și media notelor

9 Selecția datelor – interogări complexe

În această secțiune sunt prezentate interogări SQL mai complexe (cu `JOIN`, `GROUP BY`, `HAVING`, subinterogări și funcții), care pot fi utilizate și în definirea unor vederi (*views*).

9.1 Interogarea 1: Echipe ordonate după media notelor (din Evaluations)

Scop: Interogarea calculează pentru fiecare echipă:

- numărul de evaluări primite (`nr_evaluari`);
- media notelor (`medie_note`);

și afișează doar echipele care au cel puțin o evaluare. Rezultatele sunt ordonate descrescător după media notelor.

```sql
SELECT
    t.team_id,
    t.team_name,
    COUNT(e.eval_id) AS nr_evaluari,
```

```
    ROUND(AVG(e.score), 2) AS medie_note
FROM teams t
JOIN evaluations e ON e.team_id = t.team_id
GROUP BY t.team_id, t.team_name
HAVING COUNT(e.eval_id) >= 1
ORDER BY medie_note DESC;
```

TEAM_ID	TEAM_NAME	NR_EVALUARI	MEDIE_NOTE
5	EduSmart	1	9.6
3	CityCoders	1	9.1
4	HealthTechX	1	8.95
2	GreenFuture	1	8.8

Imaginea 60: Rezultatul interogării 1 (echipe + medie note)

9.2 Interogarea 2: Teme cu număr de echipe și scor mediu

Scop: Interogarea grupează echipele după temă și calculează:

- câte echipe sunt asociate fiecărei teme (`nr_echipe`);
- scorul mediu al echipelor din tema respectivă (`scor_mediu_echipe`).

Se afișează temele care au cel puțin o echipă. Rezultatele sunt ordonate descrescător după scorul mediu.

```
SELECT
    th.theme_id,
    th.theme_name,
    COUNT(t.team_id) AS nr_echipe,
    ROUND(AVG(t.score), 2) AS scor_mediu_echipe
FROM themes th
JOIN teams t ON t.theme_id = th.theme_id
GROUP BY th.theme_id, th.theme_name
HAVING COUNT(t.team_id) >= 1
ORDER BY scor_mediu_echipe DESC;
```

THEME_ID	THEME_NAME	NR_ECHIPE	SCOR_MEDIU_ECHIPE
5	Education & Learning	1	9.4
3	Smart Cities	1	9
4	Healthcare Tech	1	8.9
2	Green Energy	1	8.75

Imaginea 61: Rezultatul interogării 2 (teme + scor mediu echipe)

9.3 Interogarea 3: Liderii echipelor cu scor peste media generală

Scop: Interogarea extrage liderii echipelor (`leader_id`) împreună cu numele echipei și scorul acesteia. Filtrarea se face cu o subinterogare:

- se calculează media scorurilor tuturor echipelor (`SELECT AVG(score) FROM teams`);

- se păstrează doar echipele cu scor mai mare decât această medie.

```sql
SELECT
    p.participant_id,
    p.nume,
    p.prenume,
    t.team_name,
    t.score
FROM participants p
JOIN teams t ON t.leader_id = p.participant_id
WHERE t.score > (SELECT AVG(score) FROM teams)
ORDER BY t.score DESC;
```

PARTICIPANT_ID	NUME	PRENUME	TEAM_NAME	SCORE
5	Marinescu	Radu	EduSmart	9.4

Imaginea 62: Rezultatul interogării 3 (lideri peste media scorurilor)

9.4 Interogarea 4: Juraţi cu activitate ridicată şi note mari

Scop: Interogarea calculează, pentru fiecare jurat:

- numărul de evaluări date (`nr_evaluari`);
- media notelor oferite (`medie_note_date`).

Filtrarea se face cu `HAVING`:

- juratul trebuie să aibă cel puţin 2 evaluări;
- media notelor trebuie să fie mai mare de 8.5.

Rezultatele sunt ordonate descrescător după media notelor date.

```sql
SELECT
    j.judge_id,
    j.full_name,
    j.email,
    COUNT(e.eval_id) AS nr_evaluari,
    ROUND(AVG(e.score), 2) AS medie_note_date
FROM judges j
JOIN evaluations e ON e.judge_id = j.judge_id
GROUP BY j.judge_id, j.full_name, j.email
HAVING COUNT(e.eval_id) >= 2
   AND AVG(e.score) > 8.5
ORDER BY medie_note_date DESC;
```

```
356 SELECT
357     j.judge_id,
358     j.full_name,
359     j.email,
360     COUNT(e.eval_id) AS nr_evaluari,
361     ROUND(AVG(e.score), 2) AS medie_note_date
362 FROM judges j
363 JOIN evaluations e ON e.judge_id = j.judge_id
364 GROUP BY j.judge_id, j.full_name, j.email
365 HAVING COUNT(e.eval_id) >= 2
366    AND AVG(e.score) > 8.5
367 ORDER BY medie_note_date DESC;
```

Results | Explain | Describe | Saved SQL | His

no data found

Imaginea 63: Rezultatul interogării 4 (juraţi activi + note mari)

9.5 Interogarea 5: Idei cu tehnologii specifice și descriere suficient de lungă

Scop: Interogarea selectează idei din tabelul `ideas` împreună cu tema asociată, iar filtrarea impune:

- software-ul să conțină `python` sau `java` (indiferent de litere mari/mici);
- descrierea să aibă minim 20 caractere.

În plus, se calculează lungimea descrierii (`LENGTH(description)`) și rezultatele sunt ordonate descrescător.

```sql
SELECT
    i.idea_id,
    i.title,
    th.theme_name,
    i.software,
    LENGTH(i.description) AS lungime_descriere
FROM ideas i
JOIN themes th ON th.theme_id = i.theme_id
WHERE (LOWER(i.software) LIKE '%python%' OR LOWER(i.software)
   LIKE '%java%')
  AND LENGTH(i.description) >= 20
ORDER BY lungime_descriere DESC;
```

IDEA_ID	TITLE	THEME_NAME	SOFTWARE	LUNGIME_DESCRIERE
4	HealthAI	Healthcare Tech	Python, TensorFlow	50
2	Green Tracker	Green Energy	Java, Spring	42

2 rows returned in 0.02 seconds Download

Imaginea 64: Rezultatul interogării 5 (idei filtrate după software + descriere)

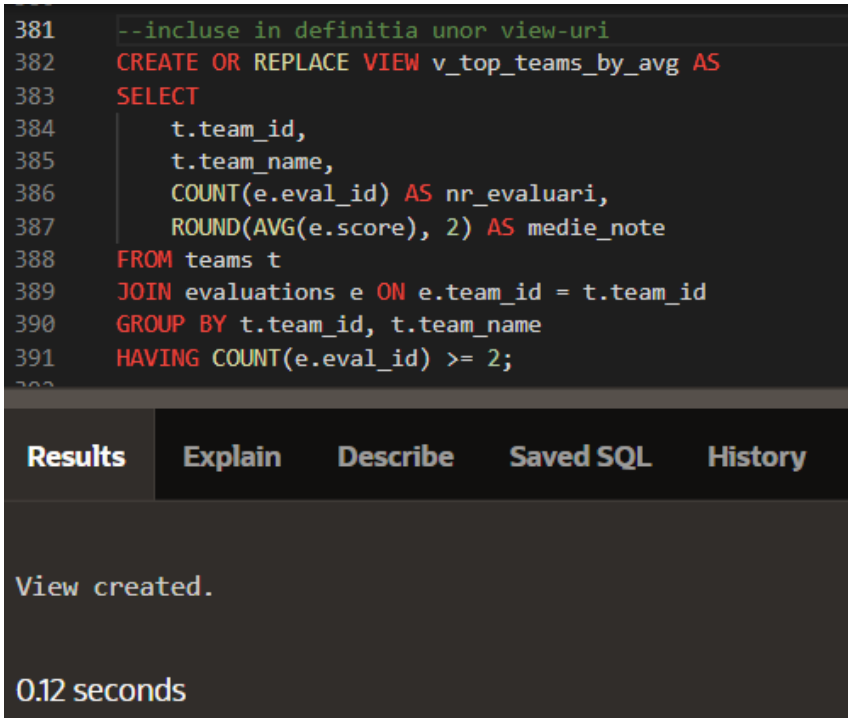
10 Interogări incluse în definiția unei vederi (VIEW)

În continuare, una dintre interogările de mai sus este inclusă în definiția unei vederi. O vedere (*view*) reprezintă o „tabelă virtuală”, definită printr-un `SELECT`, care poate fi reutilizată ulterior în alte interogări.

10.1 Crearea vederii `V_TOP_TEAMS_BY_AVG`

Scop: Vederea păstrează echipele care au minim 2 evaluări și reține:

- numărul de evaluări pentru fiecare echipă;
- media notelor.

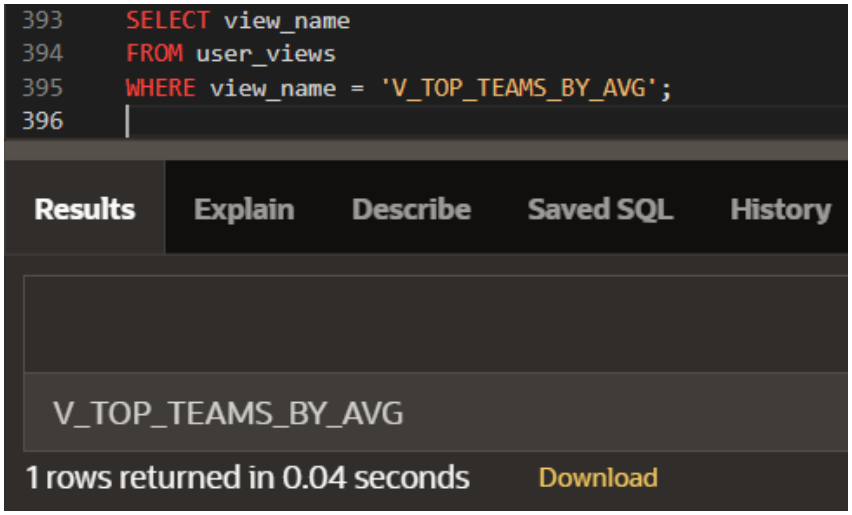


Imaginea 65: Crearea vederii `V_TOP_TEAMS_BY_AVG`

10.1.1 Confirmarea existenței vederii în dicționarul de date

Scop: Interogarea verifică dacă vederea a fost creată cu succes, folosind `USER_VIEWS`.

```
SELECT view_name
FROM user_views
WHERE view_name = 'V_TOP_TEAMS_BY_AVG';
```



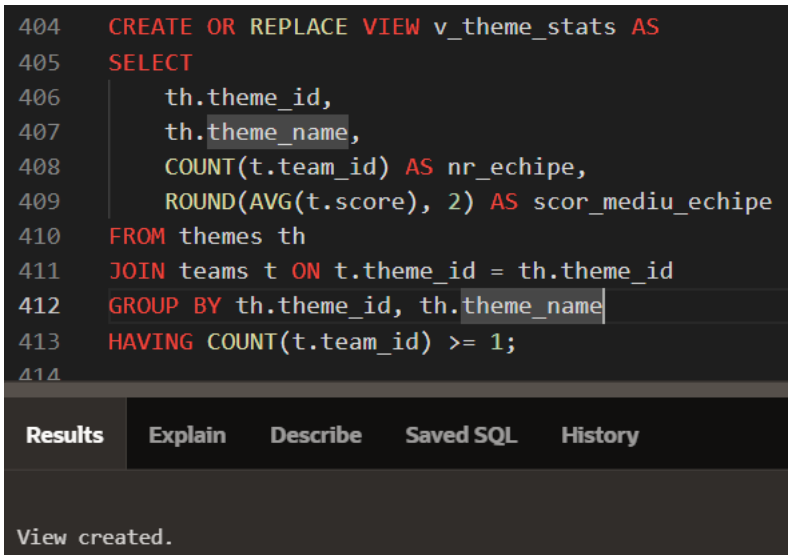
Imaginea 66: Confirmarea existenței vederii V_TOP_TEAMS_BY_AVG

10.2 Crearea vederii V_THEME_STATS

Scop: Vederea calculează pentru fiecare temă:

- numărul de echipe înscrise;
- scorul mediu al echipelor din tema respectivă.

Sunt păstrate doar temele care au cel puțin o echipă asociată.



Imaginea 67: Crearea vederii V_THEME_STATS

Pentru a vizualiza rezultatul obținut, se rulează comanda:

```
SELECT *
FROM v_theme_stats
ORDER BY scor_mediu_echipe DESC;
```

THEME_ID	THEME_NAME	NR_ECHIPE	SCOR_MEDIU_ECHIPE
5	Education & Learning	1	9.4
3	Smart Cities	1	9
4	Healthcare Tech	1	8.9
2	Green Energy	1	8.75

Imaginea 68: Rezultatul vederii V_THEME_STATS

10.3 Crearea vederii V_JUDGE_ACTIVITY

Scop: Vederea afișează juraații care au activitate ridicată, calculând:

- numărul de evaluări date;
- media notelor oferite.

Sunt păstrați doar juraații care au minim 2 evaluări și o medie a notelor mai mare de 8.5.

```
419  CREATE OR REPLACE VIEW v_judge_activity AS
420  SELECT
421      j.judge_id,
422      j.full_name,
423      j.email,
424      COUNT(e.eval_id) AS nr_evaluari,
425      ROUND(AVG(e.score), 2) AS medie_note_date
426  FROM judges j
427  JOIN evaluations e ON e.judge_id = j.judge_id
428  GROUP BY j.judge_id, j.full_name, j.email
429  HAVING COUNT(e.eval_id) >= 2
430     AND AVG(e.score) > 8.5;
```

Results Explain Describe Saved SQL History

View created.

Imaginea 69: Crearea vederii V_JUDGE_ACTIVITY

Pentru a vizualiza rezultatul obținut, se rulează comanda:

```
1 SELECT *
2 FROM v_judge_activity
3 ORDER BY medie_note_date DESC;
```

```
SELECT *
FROM v_judge_activity
ORDER BY medie_note_date DESC;
```

Results | Explain | Describe | Saved SQL | History

no data found

Imaginea 70: Rezultatul vederii V_JUDGE_ACTIVITY

10.4 Crearea vederii V_TEAM_EVAL_RANK

Scop: Vederea realizează un clasament al echipelor în cadrul fiecărei teme, afișând poziția echipei (rank) în tema aleasă, pe baza scorului.

```
CREATE OR REPLACE VIEW v_team_eval_rank AS
SELECT
    th.theme_name,
    t.team_id,
    t.team_name,
    t.score,
    DENSE_RANK() OVER (PARTITION BY th.theme_id ORDER BY t.score DESC) AS pozitie_in_tema
FROM teams t
JOIN themes th ON th.theme_id = t.theme_id;
```

Results | Explain | Describe | Saved SQL | History

View created.

Imaginea 71: Crearea vederii V_TEAM_EVAL_RANK

Pentru a vizualiza rezultatul obținut, se rulează comanda:

```
SELECT *
FROM v_team_eval_rank
ORDER BY theme_name, pozitie_in_tema, score DESC;
```

THEME_NAME	TEAM_ID	TEAM_NAME	SCORE	POZITIE_IN_TEMA
Education & Learning	5	EduSmart	9.4	1
Green Energy	2	GreenFuture	8.75	1
Healthcare Tech	4	HealthTechX	8.9	1
Smart Cities	3	CityCoders	9	1

4 rows returned in 0.02 seconds Download

Imaginea 72: Rezultatul vederii V_TEAM_EVAL_RANK